

Norihisa Ishida

Android 基礎

Java によるアンドロイドプログラミング

Norihisa Ishida

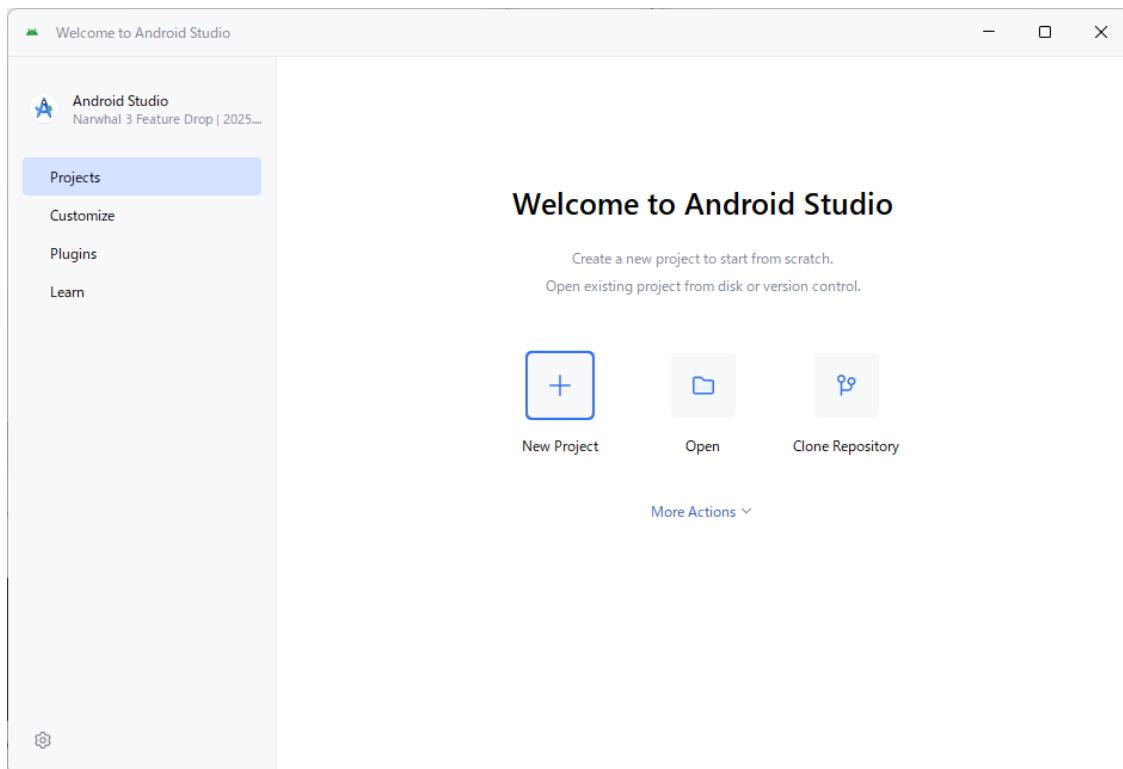
2025 年 10 月 20 日

1. プロジェクトの準備

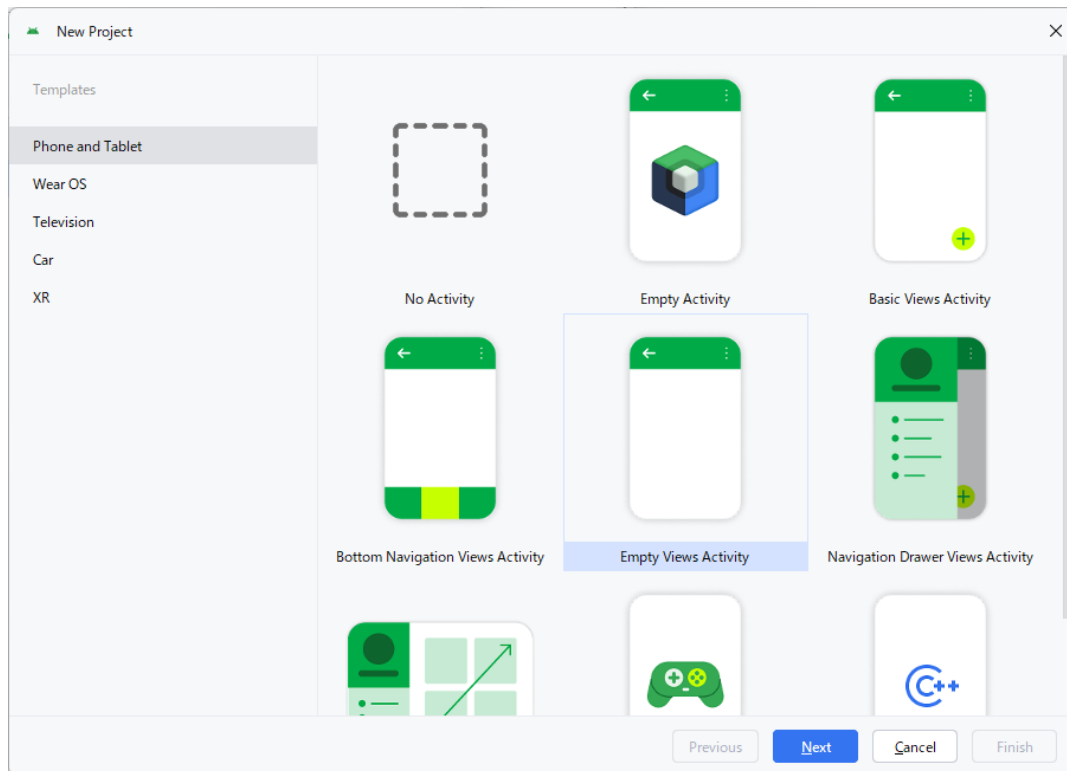
アンドロイドスタジオを起動します（以下は起動時のバナー画面）。



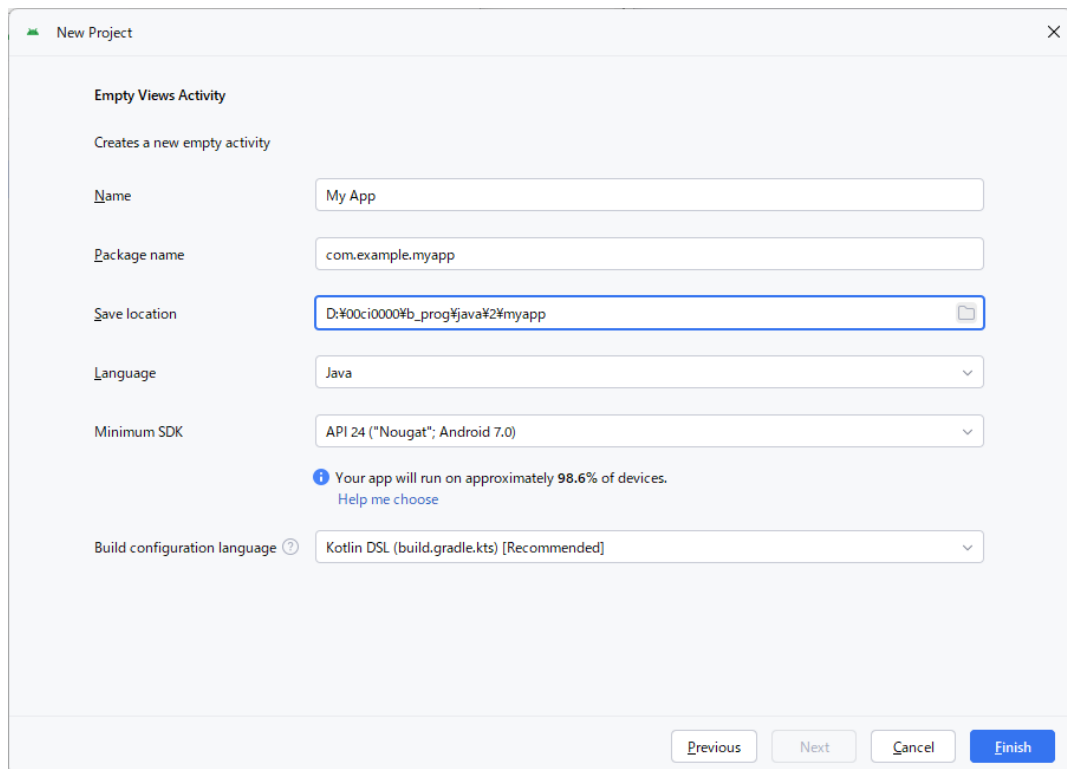
次に Welcome 画面で New Project をクリックします。



左の方で「Phone and Tablets」が選ばれていることを確認し、「Empty Views Activity」をクリックして「Next」をクリックします。

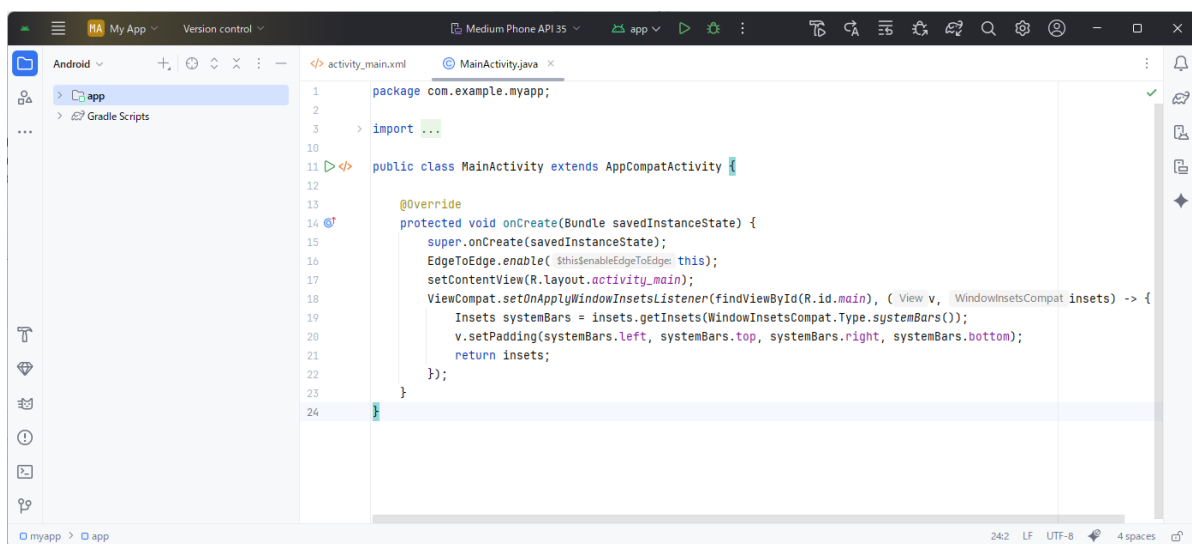


この画面が出たら「Name」の部分を「My App」としてみます。

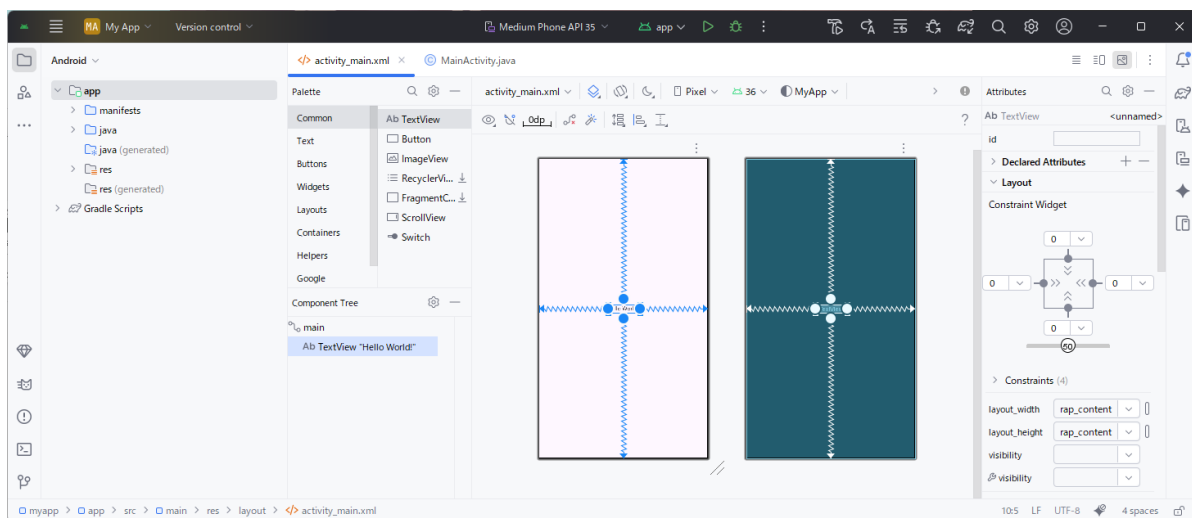


この時、Package Name と Save location の末尾が「Name」に記述した名前に自動変更されることを確認してください。次に Save Location を自身の保存場所に設定します。右側のフォルダアイコンから選択可能です。このとき、Save Location の末尾がプロジェクトフォルダになっていないので、自身でフォルダ名をここで直接書くか、あらかじめフォルダを作っておいてからそのフォルダを選択するようにします。なお Save location のパスの中に日本語や半角スペースが含まれているとちゃんと動きませんのでその場合は変更をしてください。この時は末尾が「Name」のところと同じになるように入力してください。また Language は Java に変更します。この時点で画面の下部に警告やエラーが出ていないことを確認してから「Finish」を押します。

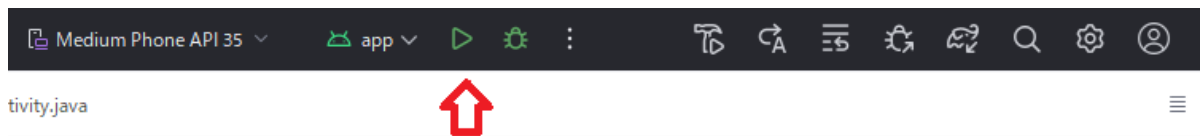
以下が起動時の IDE 画面です。この画面になるまで時間がかかりますので、右下のステータスを見ながら処理が終わるまで待ちます。



タブの「activity_main.xml」(中央上部または左側のプロジェクトウィンドウ内) をクリックすると以下のような画面にもなります。

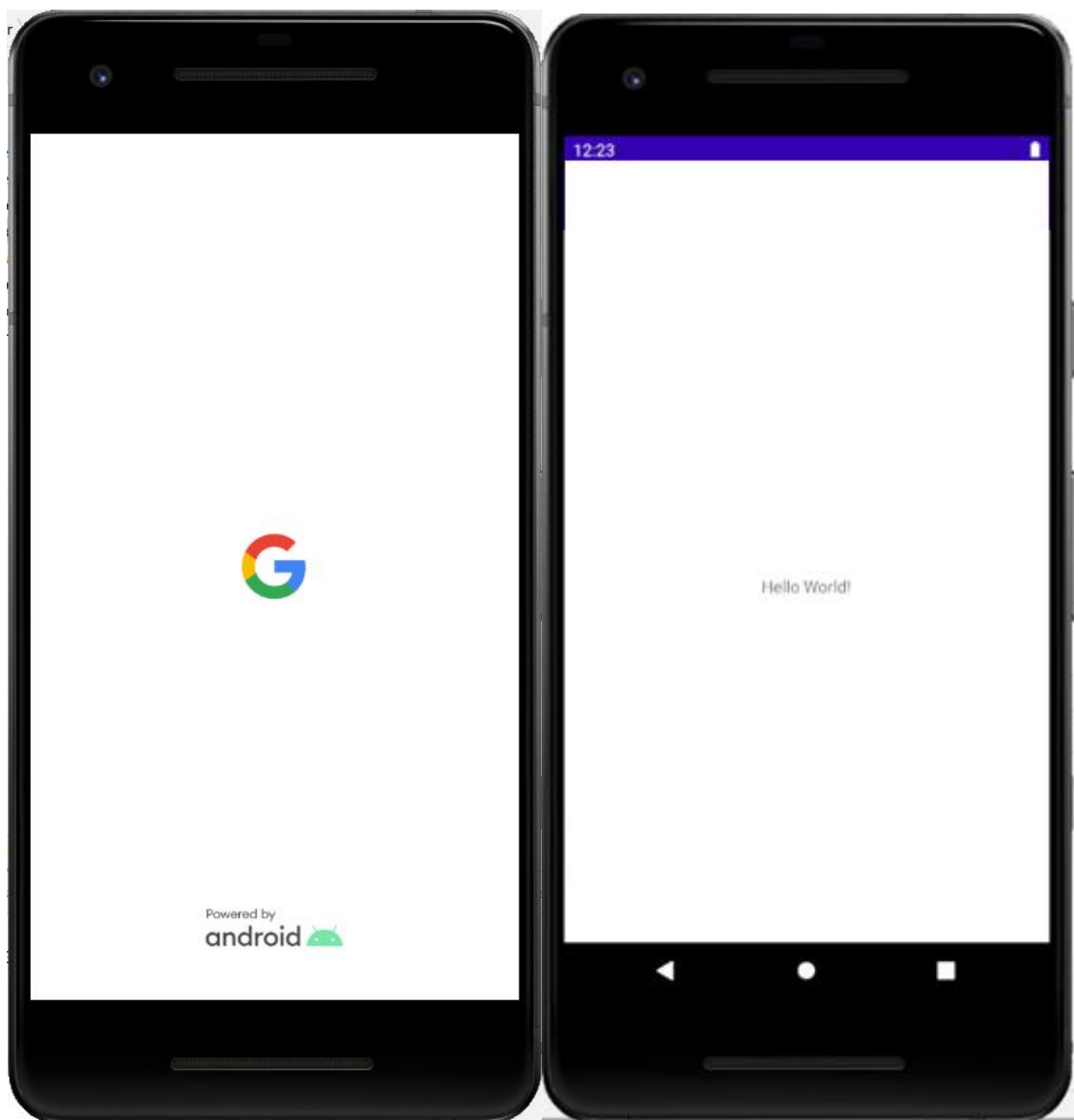


まず動作確認のため、右上の緑色の三角形（下記参照）をクリックします。

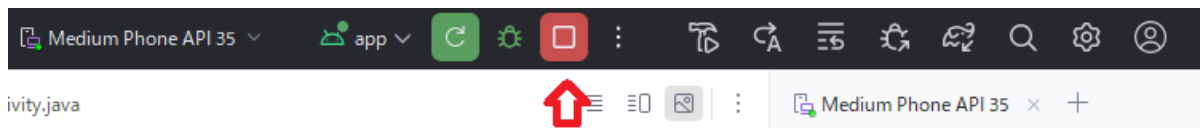


この画面では端末（Medium Phone API35）が選ばれていますが、実機を接続すると「Huawei P35」などとなります。もしこのままだとエミュレータが起動します。

起動途中の画面と実行画面。



これで動作確認は完了です。以下の赤い四角形をクリックしてプログラムを終了してください。



なお終了は端末側でもできますが、開発時は基本的に IDE 側の操作だけで操作を行います。電源 OFF など不要です。そのまま起動状態にしておいてください。

2. 最初のプログラムの確認

以下は「Empty Views Activity」テンプレートを選んだ時のスケルトン・コードです。

```
package com.example.myapplication;
import android.os.Bundle;
import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main),
            (v, insets) -> {
                Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
                v.setPadding(systemBars.left, systemBars.top,
                    systemBars.right, systemBars.bottom);
                return insets;
            });
    }
}
```

このままでも良いのですが、基礎を学ぶ上で現在不要な部分を消していきます。

```

package com.example.myapp;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

    }
}

```

「MainActivity」クラス内の「onCreate」メソッドがエントリーポイントとなりますので、その中のコードが処理内容となります。ここでは「setContentView」メソッドで「activity_main.xml」を読み込んでいます。「setContentView」メソッドは画面（Androidでは Activity という概念で扱う）に部品（コンポーネント、特にAndroidでは View という概念で扱う）を配置するのに使うものです。1つの View しか置くことができません。

3. スケルトンの改変①、TextView

XML は後回しにして、上記 onCreate メソッド内を以下のように書き換えます（上記コードの onCreate メソッドのみ）。

```

protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    //setContentView(R.layout.activity_main); //この部分はコメント
    TextView view = new TextView(this); //追加部分
    view.setText("Hello"); //追加部分
    setContentView(view); //追加部分
}

```

TextView は「TextV…」と途中まで入力すると候補が表示されますのでこれを選んでエンターを押してください。これで自動的にインポート行が追加されます。入力した状態で赤くなっている場合は、赤い文字部分をクリックしてアンダーラインが表示されたら「Alt+Enter」キーを押してインポート行を追加させます。



端末の上部にこのように出力されれば OK です。TextView は View のサブクラスです。

問題

- ① 表示内容を名前と年齢に書き換えてみよ。
- ② `view.setAllCaps(true);` を追加して文字が全て大文字になることを確認せよ。
- ③ `view.setTextAlignment(TextView.TEXT_ALIGNMENT_CENTER);` を追加して以下のようになることを確認せよ。



- ④ `view.setTextColor(Color.RED);` を追加して文字が赤くなることを確認せよ。
- ⑤ `view.setTextSize(72);` を追加して文字が大きくなることを確認せよ。
- ⑥ 「Hello」という文字を String 型の変数 `str` へ参照させて表示してみよ。以下の通り。

```
String str = "Hello";
view.setText(str);
```
- ⑦ 文字列の途中に改行のエスケープシーケンス(`\n`)を入れると改行されて表示されることを確認せよ。例えば `"Hel\nlo"` と書けばよい。

4. スケルトンの改変②、Button

今度は上記 `onCreate` メソッド内を以下のように書き換えます（上記コードの `onCreate` メソッドのみ）。

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    Button button = new Button(this);
    button.setText("Hello");
    setContentView(button);
}
```

実行して画面全体がボタンになれば OK です。Button は View のサブクラスです。

問題

- ① ボタンのラベルを自身の名前に変更せよ。
- ② `button.setRotation(45);`を追加してボタンが45度回転することを確認せよ。
- ③ `button.setAllCaps(false);`を追加してラベル文字に小文字が反映することを確認せよ。
- ④ `button.setAlpha(0.2f);`を追加してラベル文字が薄く表示されることを確認せよ。
- ⑤ `button.setTextColor(Color.BLUE);`を追加してラベル文字が青くなることを確認せよ。
- ⑥ `button.setBackgroundColor(Color.YELLOW);`を追加してボタンの文字の後ろが黄色くなることを確認せよ。
- ⑦ `button.setVisibility(View.INVISIBLE);`を追加するとボタンが表示されなくなることを確認せよ。

5. スケルトンの改変③、レイアウトの追加

上記のように View は1つしか画面に配置できないので、コンテナとなるレイアウト (View) を用意してボタンなどの View をレイアウトに張り付け、レイアウトを Activity にセットします。上の TextView と Button を用意してさらに ConstraintLayout に追加するコードにしてみます。

```
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    Button button = new Button(this);  
    button.setText("Hello");  
    TextView view = new TextView(this);  
    view.setText("World");  
    ConstraintLayout layout = new ConstraintLayout(this);  
    layout.addView(button);  
    layout.addView(view);  
    setContentView(layout);  
}
```

するとテキストとボタンが共存できますが、重なってしまっていることがわかります。ボタンの下に World の文字がかすかに見えます。



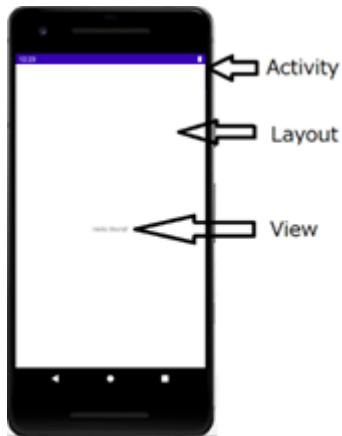
もしボタンとテキストを重ねないようにしたければ補足 A のようにします。ただしコードの可読性が低く分かりにくいいため、一般的には次の XML を使うことが有効となります。

問題

- ① `view.setPadding(250,50,0,0);`を追加して、文字がボタンの右側に表示されることを確認せよ。
- ② `TextView` をもう一つ用意して、ボタンの下に自身の名前を表示してみよ。

6. XML の利用

複数の `View` を画面に配置するためにはレイアウトなどのコンテナをまず用意してその上にボタンなどの `View` を張り付けることで、`Activity` にはレイアウトだけセットして複数の部品が画面に配置できることがわかりました。



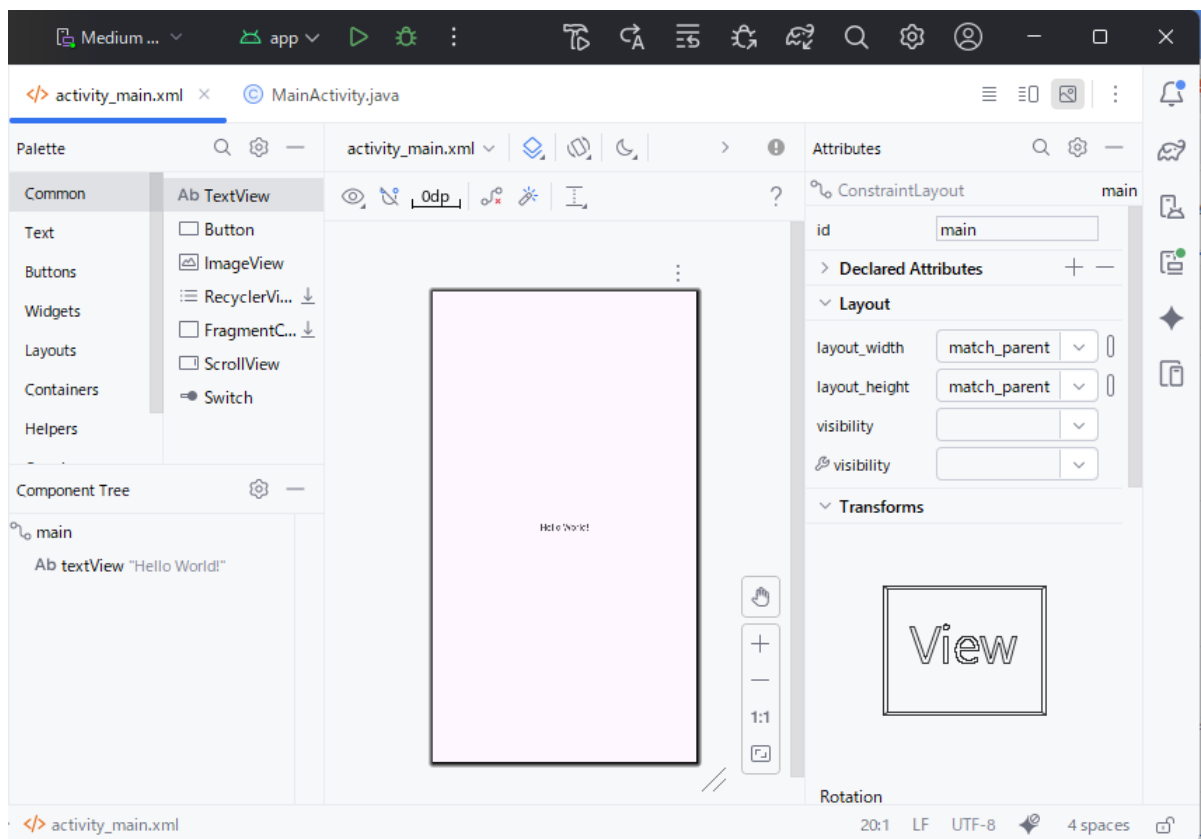
ただしこの方法は位置を指定したりそれぞれのコンテナとコンポーネントの関係を結ぶ処理を記述したりなど、コードで記述するのは直感的ではありません。このまま複雑な画面構成やデザイン性に優れた UI を試行錯誤とともに構築するのはかなり大変です。そこで XML スクリプトを利用します。XML はドキュメントフォーマットで Web で使われていますが、XML のコードそのものを編集するのではなく、XML 用のデザインツールを用いて編集します。基本的にはマウスでの操作で、プレビュー画面を直接編集しますので直感的です。最初のコードに戻して確認します。新規にプロジェクトを作ったほうが良いと思います。コード部分は最初のものになりますので、`setContentView` は以下のようになっているはずです。

```
setContentView(R.layout.activity_main);
```

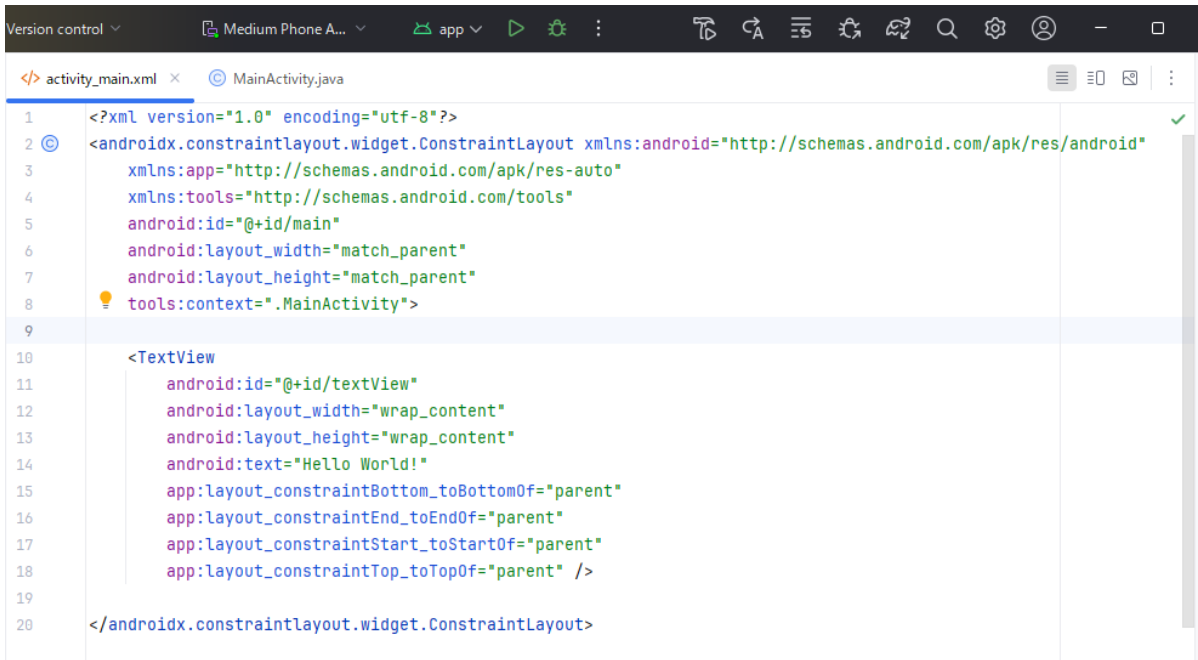
ちなみに XML などコード以外の要素をリソースと言い、アンドロイドではユニーク番号を割り振って各リソースをプログラム上で認識させる仕組みになっています。このユニーク番号を管理する R.java はアンドロイドが自動的に番号を割り振るために編集不可で、アンドロイドスタジオから見ることもできません。引数の部分は「R クラスの中の layout クラス内の static 定数の activity_main」という意味です。すべて static メンバです。

XML の方を見てみます。最初から ConstraintLayout が配置されていることがわかります。デザイン画面では左下の Component Tree の部分、コード画面では 2 行目の記述です。

デザイン画面：

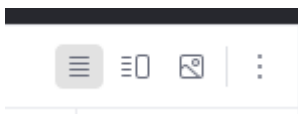


コード画面：

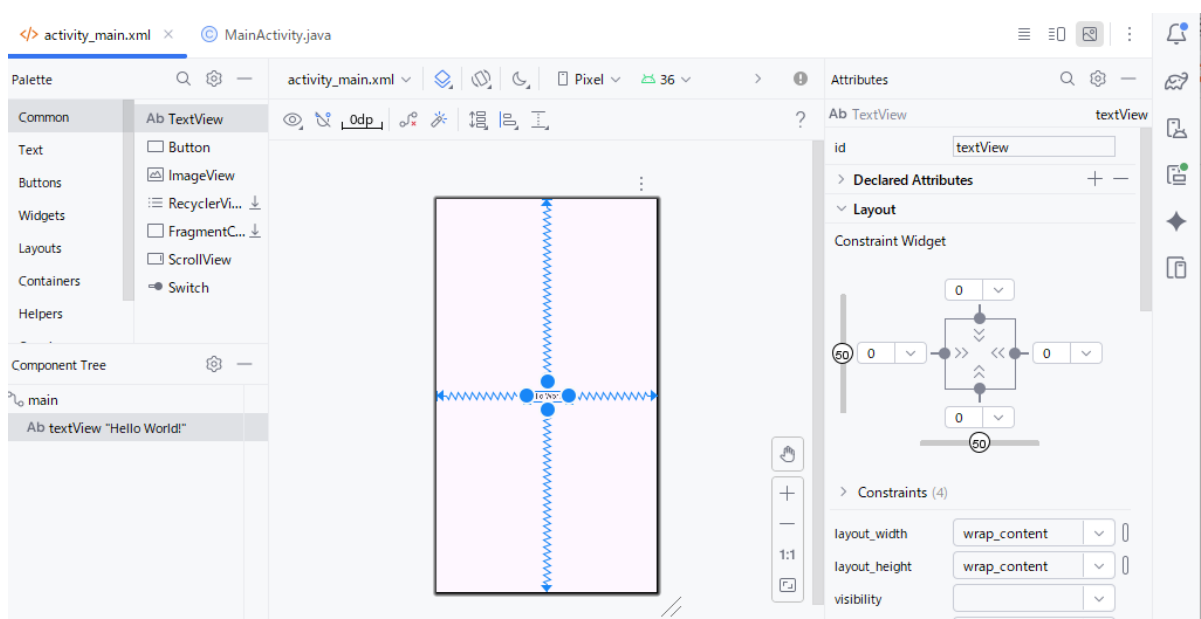


```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:app="http://schemas.android.com/apk/res-auto"
4     xmlns:tools="http://schemas.android.com/tools"
5     android:id="@+id/main"
6     android:layout_width="match_parent"
7     android:layout_height="match_parent"
8     tools:context=".MainActivity">
9
10     <TextView
11         android:id="@+id/textView"
12         android:layout_width="wrap_content"
13         android:layout_height="wrap_content"
14         android:text="Hello World!"
15         app:layout_constraintBottom_toBottomOf="parent"
16         app:layout_constraintEnd_toEndOf="parent"
17         app:layout_constraintStart_toStartOf="parent"
18         app:layout_constraintTop_toTopOf="parent" />
19
20 </androidx.constraintlayout.widget.ConstraintLayout>
```

ちなみにデザイン画面とコード画面の切り替えは右上のボタンで切り替えます。

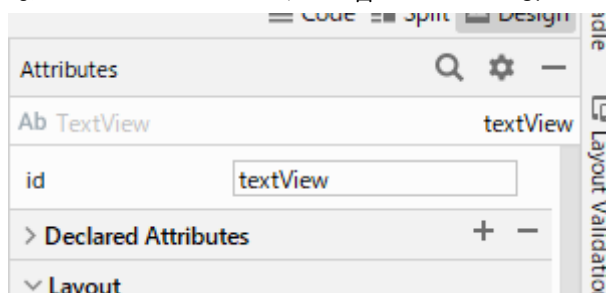


XML 側は必要な時に見るとして、デザイン画面で真中にある「Hello World」の文字をクリックしてみます。もしこれで選択できないときは、左下の Component Tree にぶら下がっている TextView をクリックします。

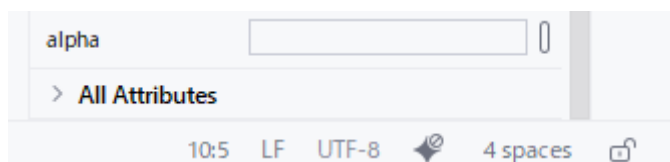


するとデザイン画面の TextView から上下左右に青いギザギザの線が 4 つ現れます。これが制約 (Constraint) です。この線はアンカーと呼ばれ TextView をクリックして動かすと伸び縮みすることがわかります。これは上下左右からの位置をアンカーで決めていることになります。位置の操作は右側の Layout の Constraint Widget 画面で左側と下側にスクロールバーがあり、これを動かしてパーセンテージで位置を決めることもできます。上限左右の数字は固定範囲でこの数字を大きくすると、上下左右の移動可能範囲が狭まります。内側の四角の上下左右の●はクリックすると制約を解除できます。もう一度クリックすると固定位置でアンカーが設定されます。

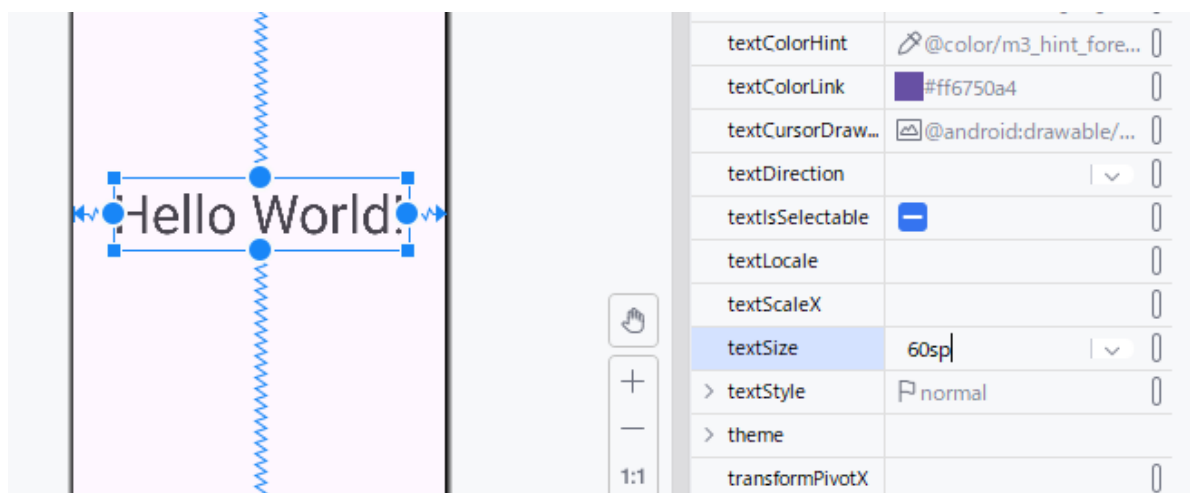
なおこの TextView には id 名がついていません (右上)。「textView」とつけておきます。



下の方をスクロールしていくと、「>All Attributes」が見えるので左の「>」をクリックしてさらにスクロールしていきます。



文字の色やサイズなどを変えられることがわかります。例えば textSize を 60sp にすると図のように文字が大きくなります。



もしこの処理をコードで行いたい場合は先ほど決めた id 名を使って以下のようなコードを追加

します。

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    TextView textView = findViewById(R.id.text_view); //この行を追加
    textView.setTextSize(60);                        //この行を追加
}
```

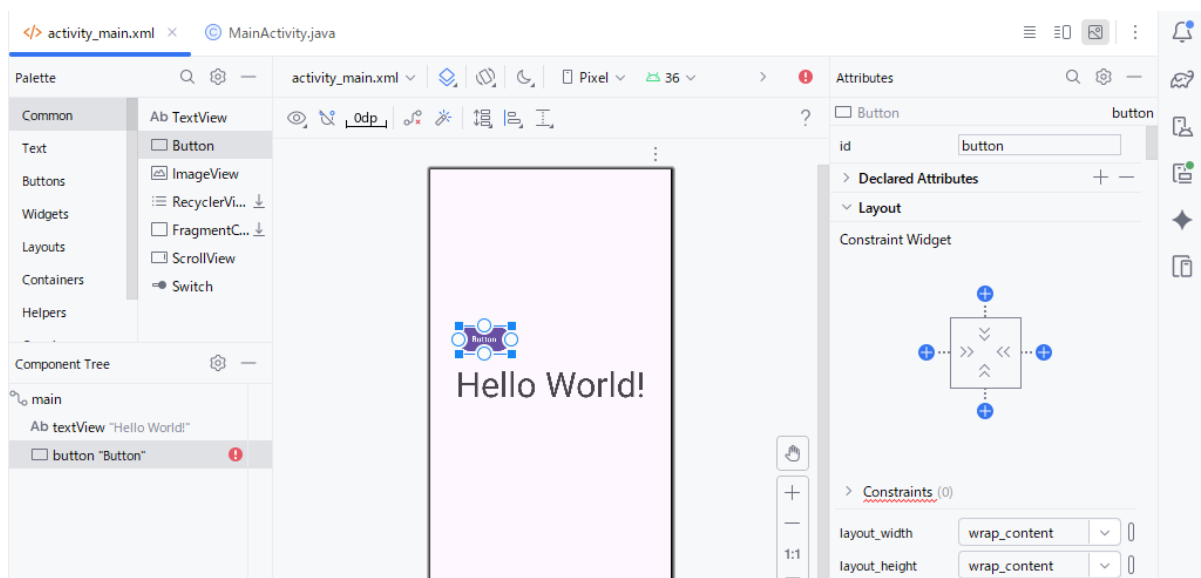
3章ではTextViewのインスタンス生成を行いました、このようにid名を使ってXML側のドキュメントオブジェクトを呼び出してインスタンスと同じように扱えることが分かりました。

問題

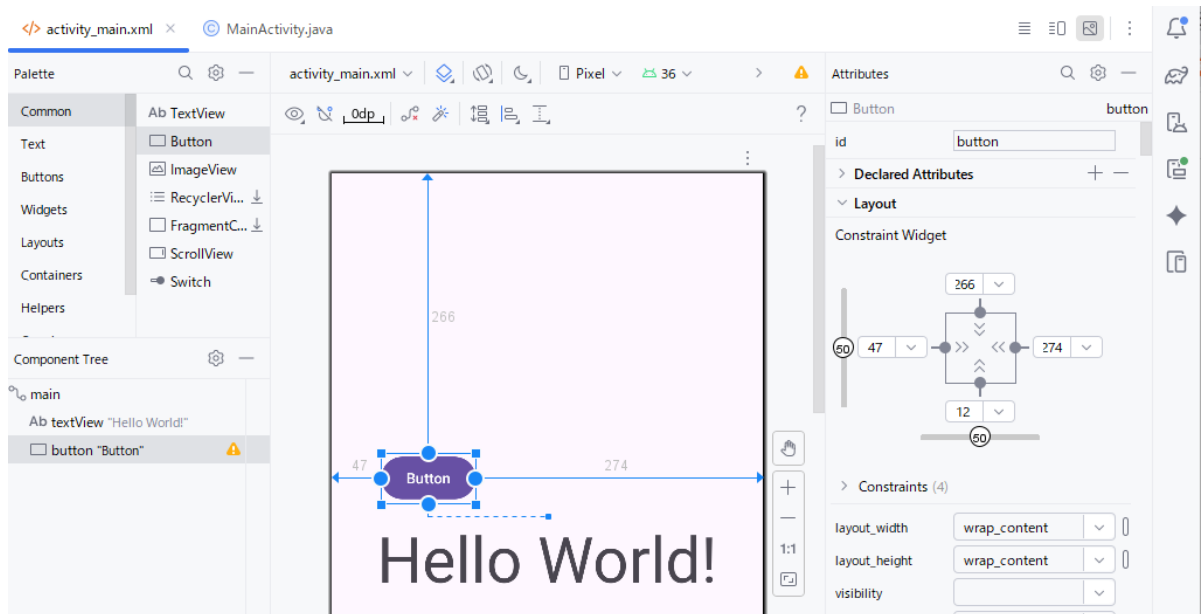
- ① 文字の色をAttribute(属性)設定を使って赤にしてみよ。
- ② 文字の色をコード上から青にしてみよ。

7. XML デザイン画面の Palette の活用

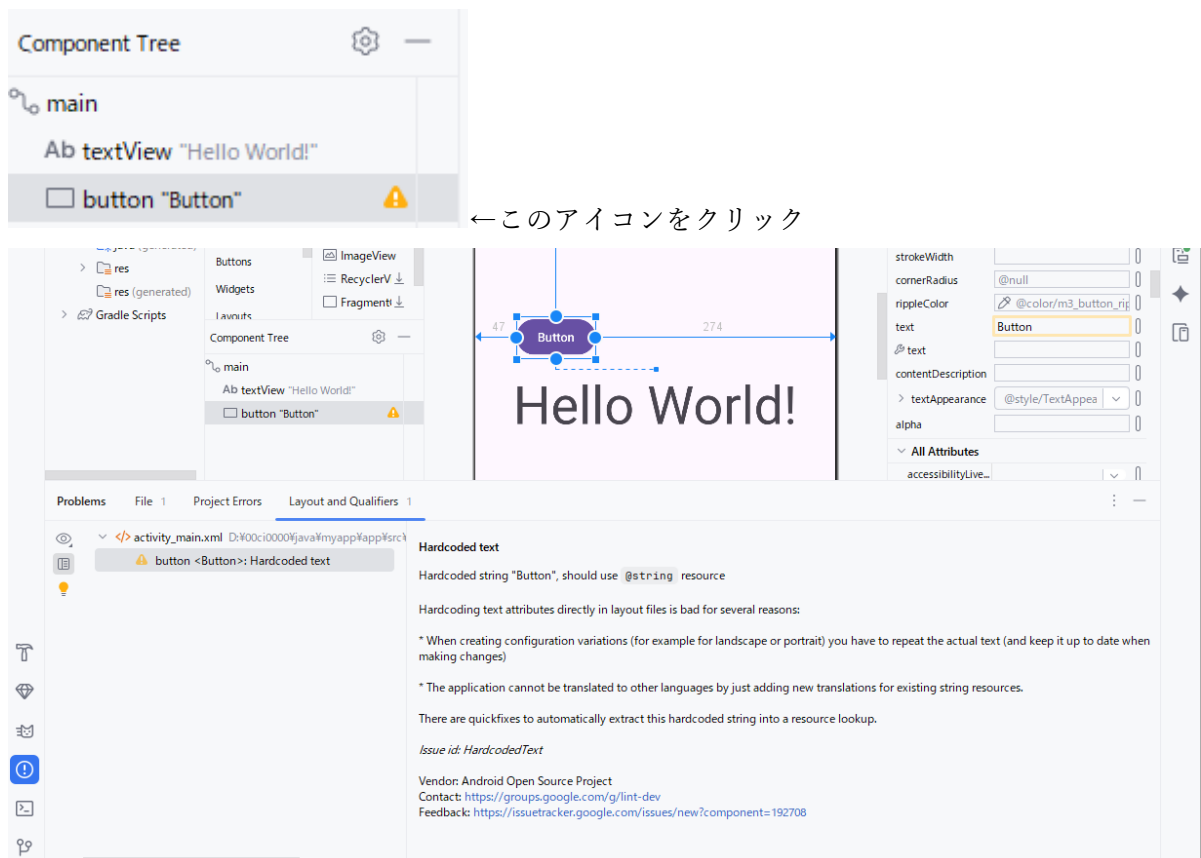
デザイン画面の左上には Palette があります。ここからボタンなどを選んでデザインに反映できます。Common の Button（または Buttons の Button）をクリックしながら Hello World と書かれた TextView の近くにドラッグして手を放します。



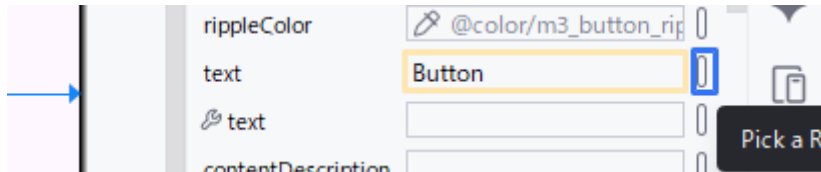
すると Component Tree の右側や Attributes の Constraint Widget の下側にエラーのような表示が現れます。これはまだ制約がつけられていないために出ますので右側の Constrain Widget 内の4か所の青い⊕をクリックして位置を固定します。



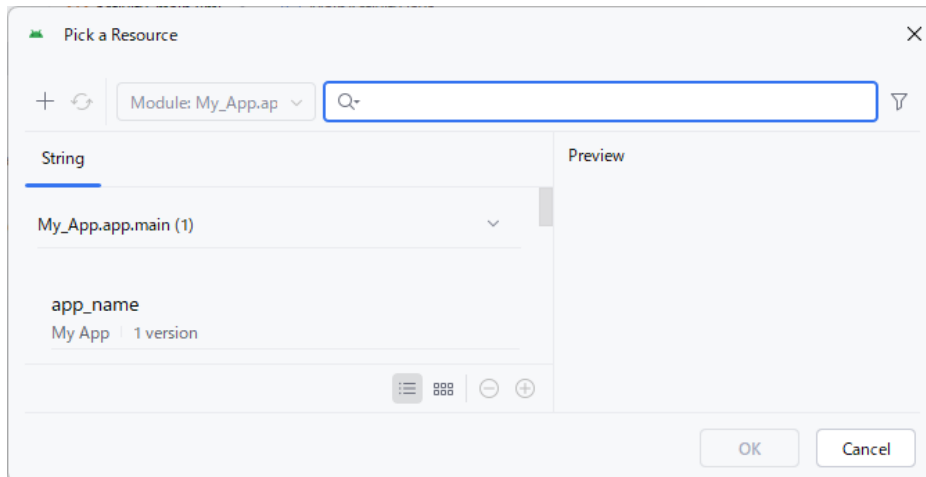
この時、Component Tree の Button の左側に黄色い三角形がついている場合はそのまま無視しても大丈夫ですが、このアイコンをクリックすると下部にメッセージが現れます。



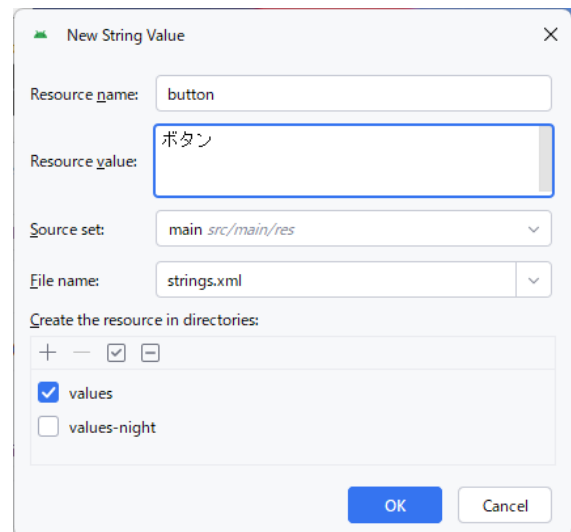
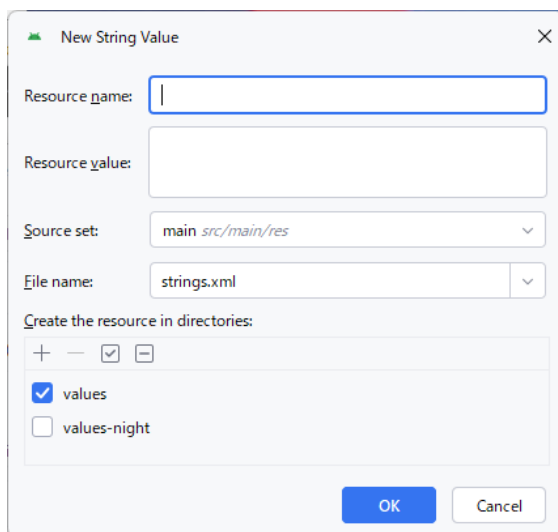
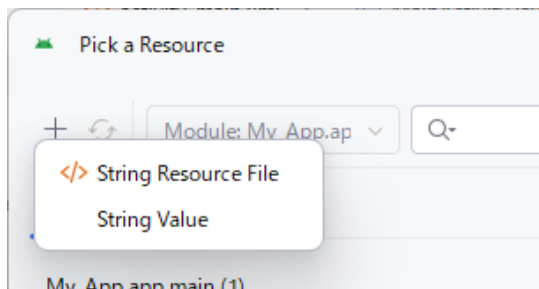
右の Attributes からスクロールして「text」を見つけ、「Button」と書かれた右の縦長の丸をクリックします。



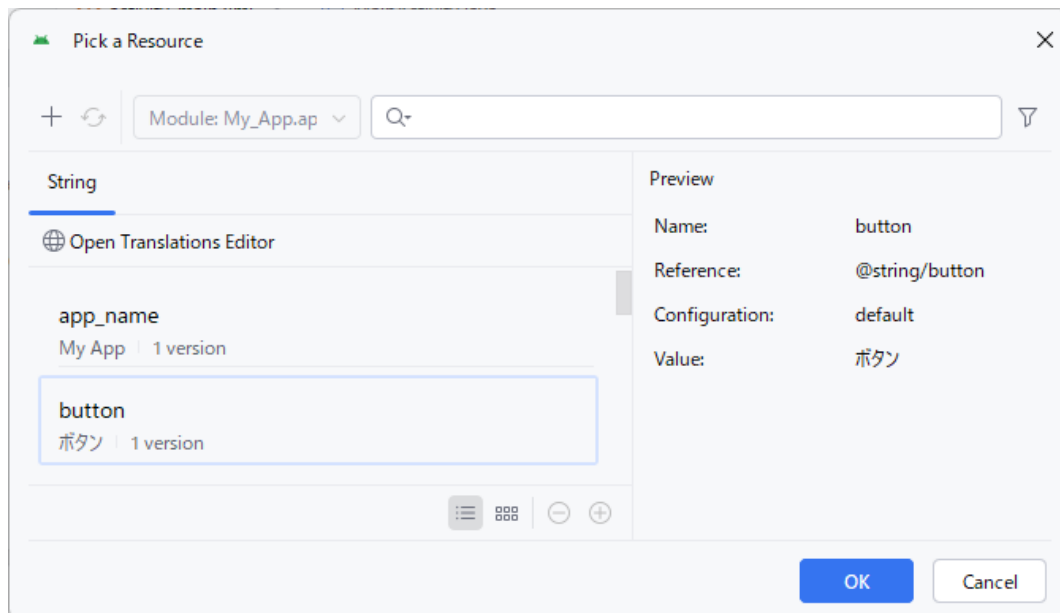
するとダイアログ画面がでますので、左上の「+」をクリックします。



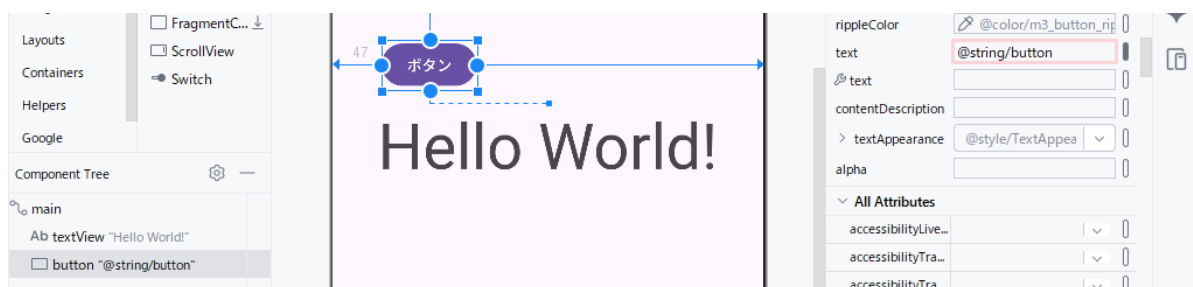
そして「String Value」を選択します。



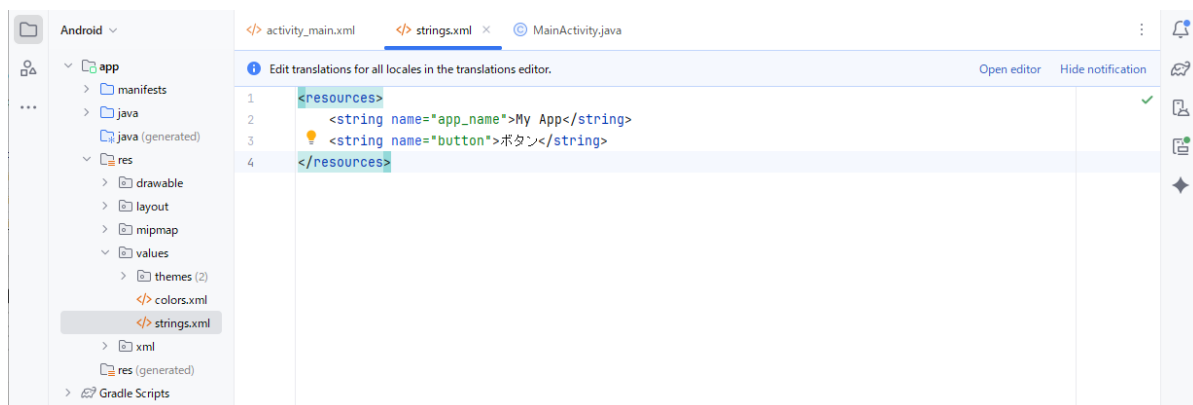
すると上記のようなダイアログがでますので図のように Resource name と Resource Value を入力して「OK」を押します。上から 2 番目の Resource Value はボタン上のラベルなので日本語も OK です。そして下の画面になったら「OK」を押します。



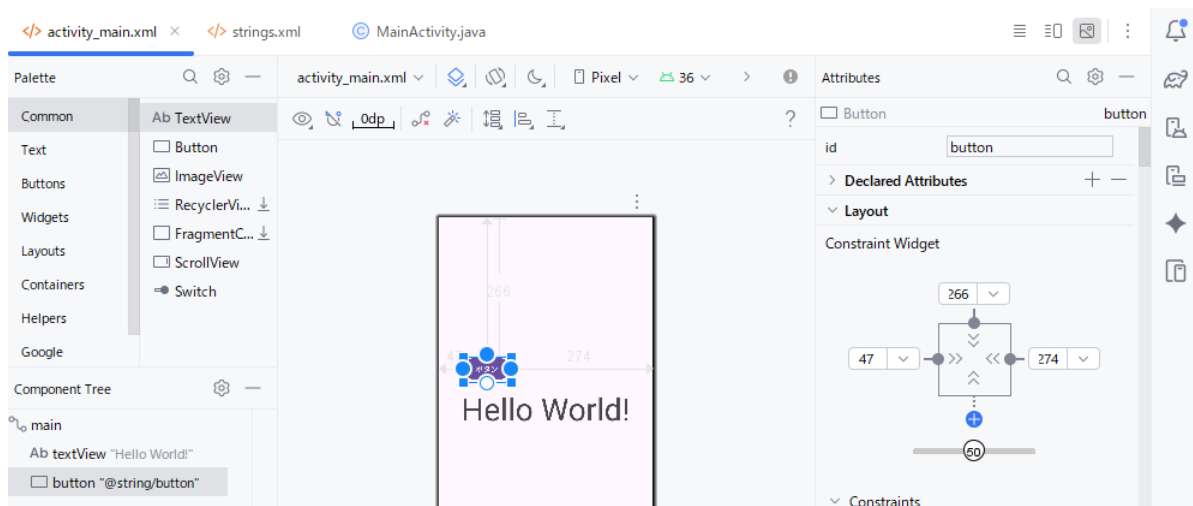
結果的に黄色い警告アイコンは消え、「Attributes」の「text」にはピンクの枠線が囲われます。



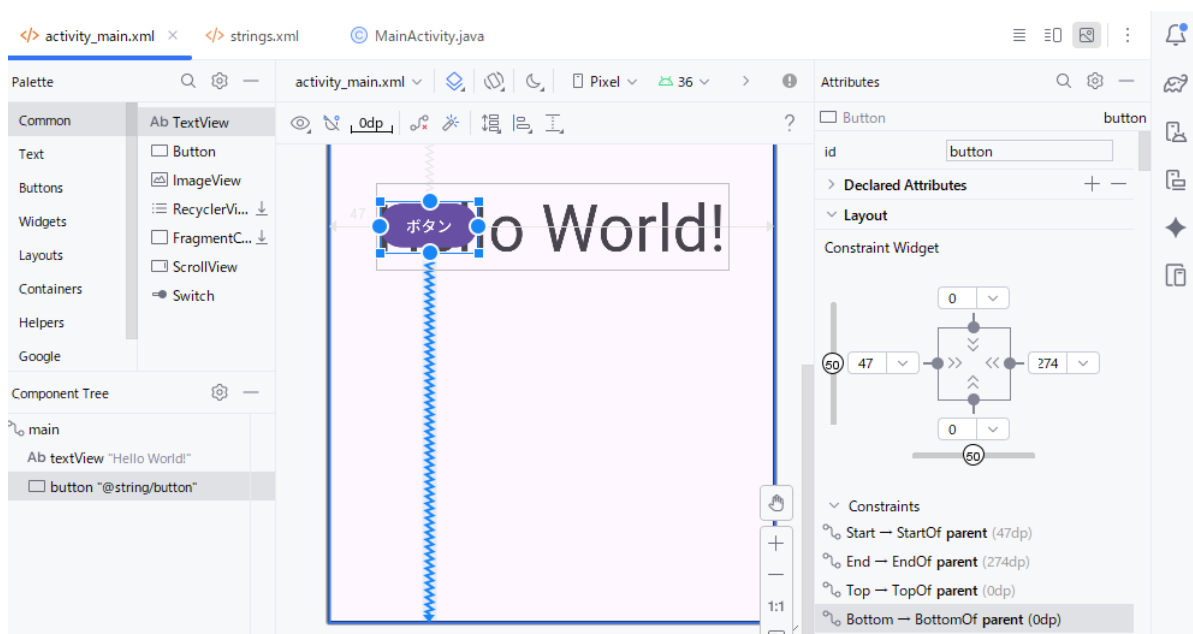
ちなみにこの作業はボタンに直接文字を記した状態（Hard Coded）からリソースにラベル文字列を登録して呼び出す方法に切り替える作業をしています。プロジェクトツリーの「res」の「values」の「strings.xml」にその記述が移ったことが下の図からわかります。



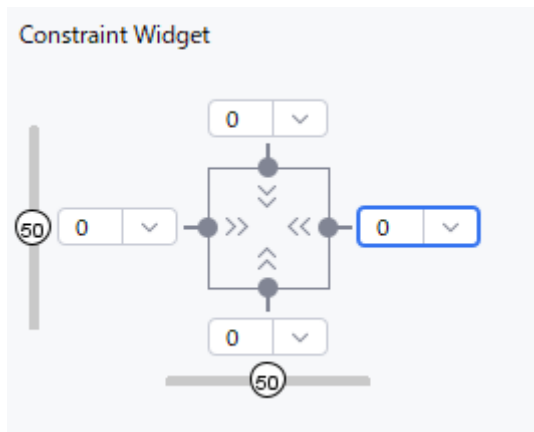
さて Constraint に戻りますが、ボタンの上下左右に出ている線のうち、下の線はすぐ下の「Hello World」の文字列につながっていますので、文字列を移動させるとボタンも追従して動いてしまいます。この状態でもいいのですが、画面の絶対位置にする場合は先ほどクリックした⊕ ボタンの下部分のみをクリックして Constraint を解除後、マウスでデザイン画面のボタンの下部に見える丸い部分をクリックしながらドラッグし、画面底部まで引っ張って手を放します。下は解除直後の画面



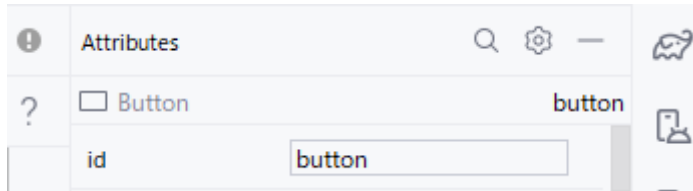
下はマウスで下まで引っ張って手を離したときの画面



あとはマウスをクリックしながらドラッグすればボタンを上下に移動できます。この時左右には動かさせませんが、右側の Constraint Widget の左右の四角内の数字を 0 にすれば動かすことができます。



ボタンの作業は以上です。この時 Attributes の「id」が「button」になっていることを確認します。もし空であれば入力しておきます。id 名を変えることもできますが、日本語は使えません。



ここから MainActivity.java の方へ移動します。先ほど TextView を追加したコードに以下のようコードを追加します。

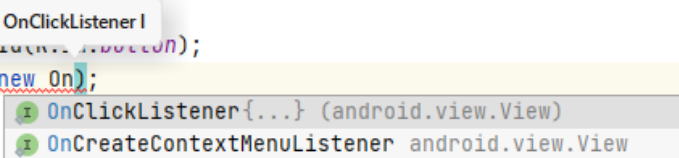
```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    TextView textView = findViewById(R.id.textView);
    textView.setTextSize(60);

    Button button = findViewById(R.id.button);           //この行を追加
    button.setOnClickListener(new View.OnClickListener() { //この行を追加
        @Override                                       //この行を追加
        public void onClick(View view) {                //この行を追加
            textView.setText("BUTTON PUSHED");          //この行を追加
        }                                                //この行を追加
    });                                                 //この行を追加
}
```

入力途中で以下のような選択画面が出たらエンターキーを押します。すると自動入力されます。むやみにすべてを入力せず、自動入力機能を使ってください。

エンター押下直前：

```
setContentView(R.layout.activity_main),
TextView textView = findViewById(R.id.textView);
textView.setTextSize(60);
Button button = findViewById(R.id.button);
button.setOnClickListener(new On);
}
```



エンター入力後：

```
setContentView(R.layout.activity_main),
TextView textView = findViewById(R.id.textView);
textView.setTextSize(60);
Button button = findViewById(R.id.button);
button.setOnClickListener(new OnClickListener() {
    @Override
    public void onClick(View view) {
    }
});
}
```

これで実行し、ボタンを押して文字列が「BUTTON PUSHED」と変わることを確認します。

問題

- ① この用例でボタンを押すとボタンが消えるようにしてみよ。ボタンを消すにはボタンクラスの `setVisibility` メソッドを使い、引数を「`View.INVISIBLE`」にします。以下の通り。

```
button.setVisibility(View.INVISIBLE);
```

- ② ボタンをもう一つ用意して一方を押すともう一方が消えるようにしてみよ。
- ③ 上の問題でボタンをもう一度押すとボタンが復活するようにしてみよ。ボタンを表示させるには同じメソッドを使い、引数を「`View.VISIBLE`」にします。1度目と2度目の判定は変数を用意して `if` 文で判定するか、配列を使って切り分けるかなど方法は問いません。
- ④ `ProgressBar(Horizontal)` を用意して、一方のボタンを押すと `ProgressBar` のメモリが 10% ずつ増え、もう一方のボタンを押すと 10% ずつ減るようにしてみよ。なお、メモリの値を決めるには `setProgress` メソッドの引数に数値(0 ~ 100)を指定します。以下。

```
progressBar.setProgress(30);
```

- ⑤ 上の問題で、バーの表示に合わせて TextView の部分にその数値を表示せよ。この時例えば「40%」のように表示し、0 以下、100 以上の値は表示しないようにすること。変数値を使ってもいいが ProgressBar クラスの `getProgress` メソッドの戻り値を使うのが良い。例えば以下のような感じで値を取得する。

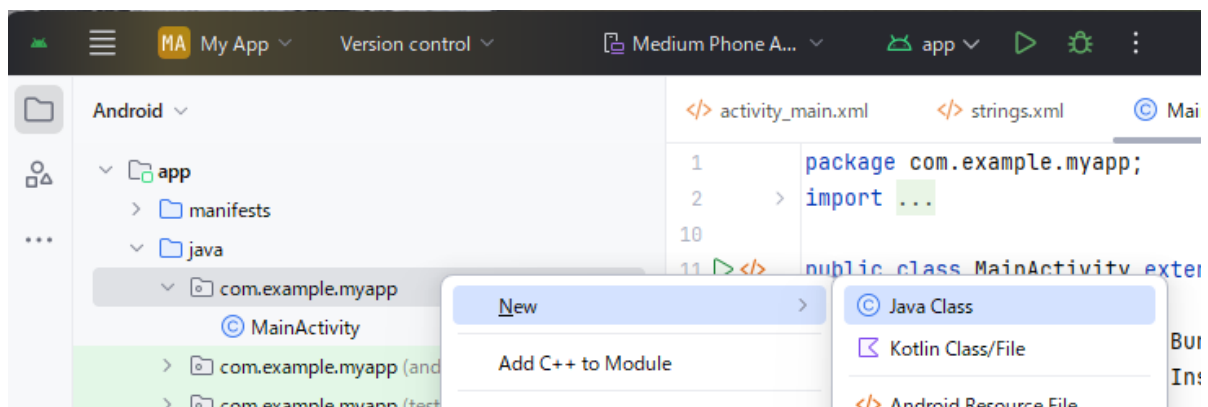
```
int value = progressBar.getProgress();
```

- ⑥ 上の問題で表示が 100%になったときは文字の表示を青、0%になったときは赤になるようにしてみよ。

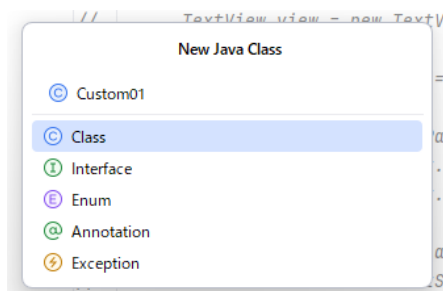
8. カスタムビュー

Button や TextView のような完成したビューを使ってアプリケーションを作れるのであれば Palette から呼び出して使うだけでいいのですが、ゲームなどを作る場合やオリジナルデザインの機能を使いたい場合は View を自作しなければなりません。パレット上の部品はすべて View クラスを継承して作られています。同様に View クラスを継承して自作の View を作ることができます。これをカスタムビューと呼びます。

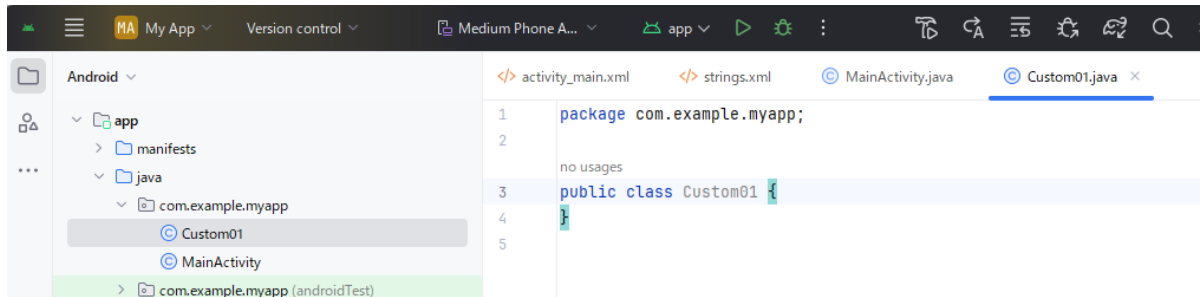
新規にソースコードを追加するには左側のプロジェクトツリーで「java」のパッケージを右クリックし、「New」→「Java Class」を選びます。



この時出てきたダイアログボックスの上部にクラス名（この例では Custom01）を入力し、下で「Class」が選択されていることを確認してエンターキーを押します。



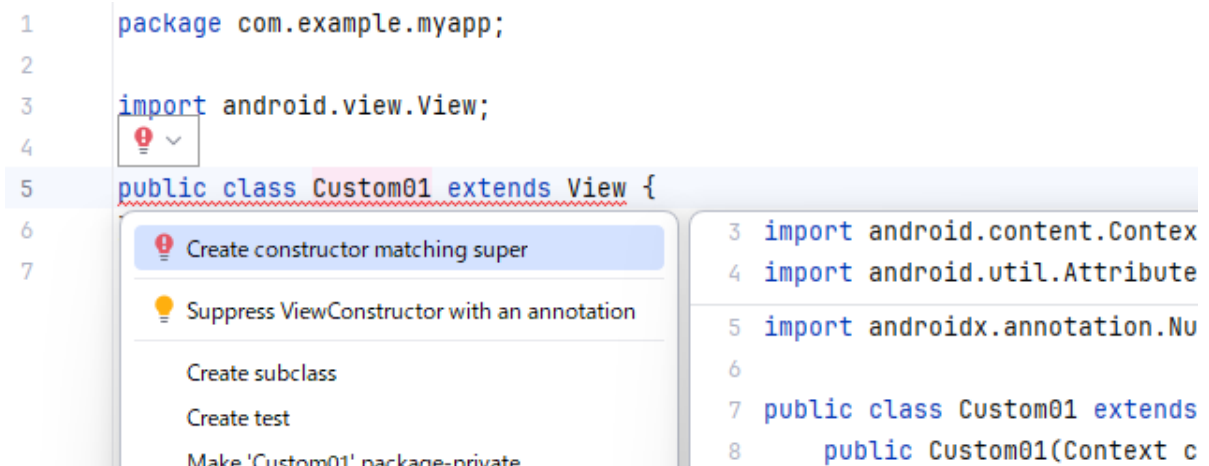
これでプロジェクトツリーに Custom01 クラスがぶら下がり、Custom01 クラスの編集ができるようになりました。



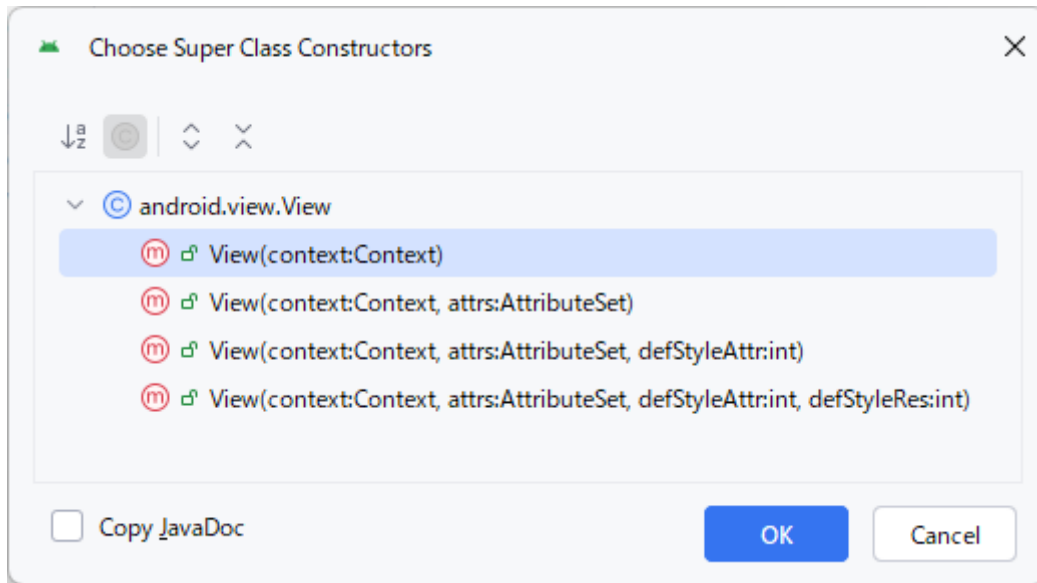
次にコードのクラス名(Custom01)の後ろに「View」クラスを継承させるための記述を追加します。このとき View クラスのインポートが追加されるよう、前述の入力補助機能などを活用しましょう。

```
1 package com.example.myapplication;  
2  
3 import android.view.View;  
4  
5 public class Custom01 extends View {  
6 }  
7
```

入力後、図のように赤い波線と赤い電球が点ります。もし電球が出ないときは赤い波線をクリックします。



ここでは一番上の「Create constructor matching super」(スーパークラスと一致するコンストラクタを生成)を選びます。



この画面では4択となりますが、単体で使用する場合は一番上、XMLと連携する場合は2番目以降を選択します。ここではまず一番上を選ぶことにします。これで準備完了です。以下。

```
package com.example.myapplication;

import android.content.Context;
import android.view.View;

public class Custom01 extends View {
    public Custom01(Context context) {
        super(context);
    }
}
```

これがカスタムビューですが、このままでは何の機能もありません。以下のようにコードを追加してみます。CanvasとPaintはインポートが必要になりますので入力補助機能を活用してください。

```
public class Custom01 extends View {
    public Custom01(Context context) {
        super(context);
    }

    @Override
    //この行を追加
```

```

public void onDraw(Canvas canvas){           //この行を追加
    Paint paint = new Paint();                //この行を追加
    paint.setColor(Color.RED);                 //この行を追加
    canvas.drawCircle(200,200,100, paint);    //この行を追加
}
}

```

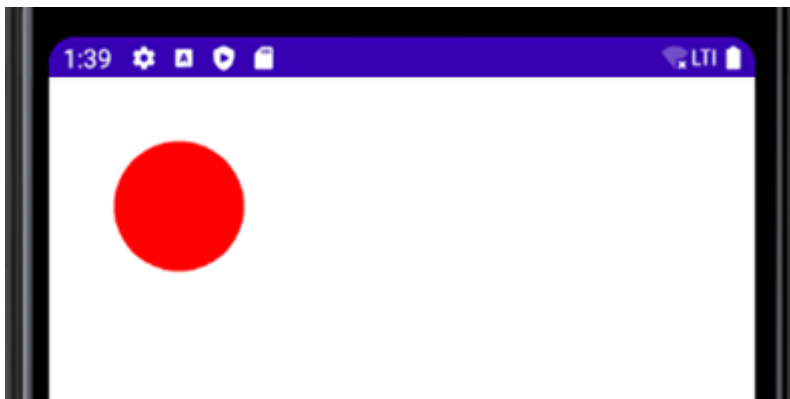
次に MainActivity クラスを書き換えます。

```

protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    //setContentView(R.layout.activity_main); //この行をコメント
    setContentView(new Custom01(this));      //この行を追加
}

```

以下のように実行されれば OK です。

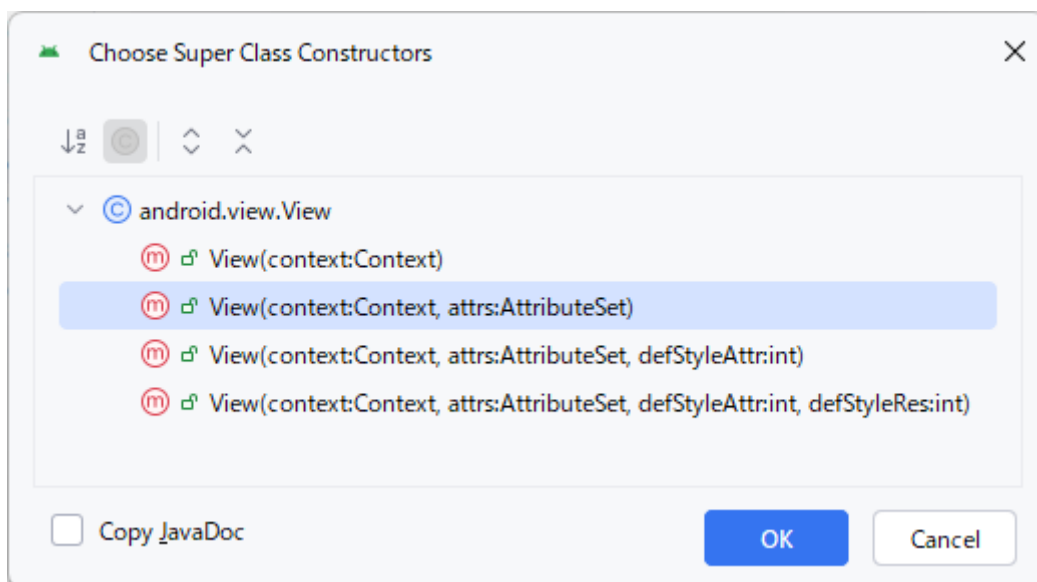


問題

- ① Custom01 クラスを改変して色を赤から青に変えてみてください。青は「Color.BLUE」です。
- ② Custom01 の円の描画に加えて自身の名前を表示してみよ。次のコードを追加すればよい。
`canvas.drawText("名前", 50, 350, paint);`
- ③ 上の問題で文字を大きくしてみよ。次のコードを drawText メソッドの上に追加すればよい。
`paint.setTextSize(72);`
- ④ 文字の色を緑色にしてみよ。円の色は変えないこと。

9. カスタムビューと XML との連携

setContentView メソッドの引数は 1 つだけなので、上記の方法は画面のなかにカスタムビューを 1 つだけ表示する方法になります。このままでは複数のカスタムビューを使うことも、ボタンなどの別の View との共存もできません。そこでカスタムビューを XML 側で登録して使う方法をとります。XML ではパレットに部品があり、そこからボタンやテキストをドラッグ&ドロップして XML のデザイン画面に貼り付けましたが、カスタムビューも同じ方法で貼り付けることができます。まずカスタムビューを以下に示すコードのようにします。ここでのクラス名は Custom02 です。前のものとの違いはコンストラクタの引数の違いだけです。まずプロジェクトツリーで MainActivity と同一のパッケージに新たに **Custom02** クラスを追加し、View クラスの継承の記述のあと、波線の左側の電球のところで「Create constructor matching super」を選んだあと、上から 2 番目を選びます。ここまでの細かな手続きは前章を確認してください。



以下のようなコードになりますが、このままでは結果が分からないので上の例と同様に描画処理を入れておきます。

```
package com.example.myapplication;

import android.content.Context;
import android.util.AttributeSet;
import android.view.View;
```

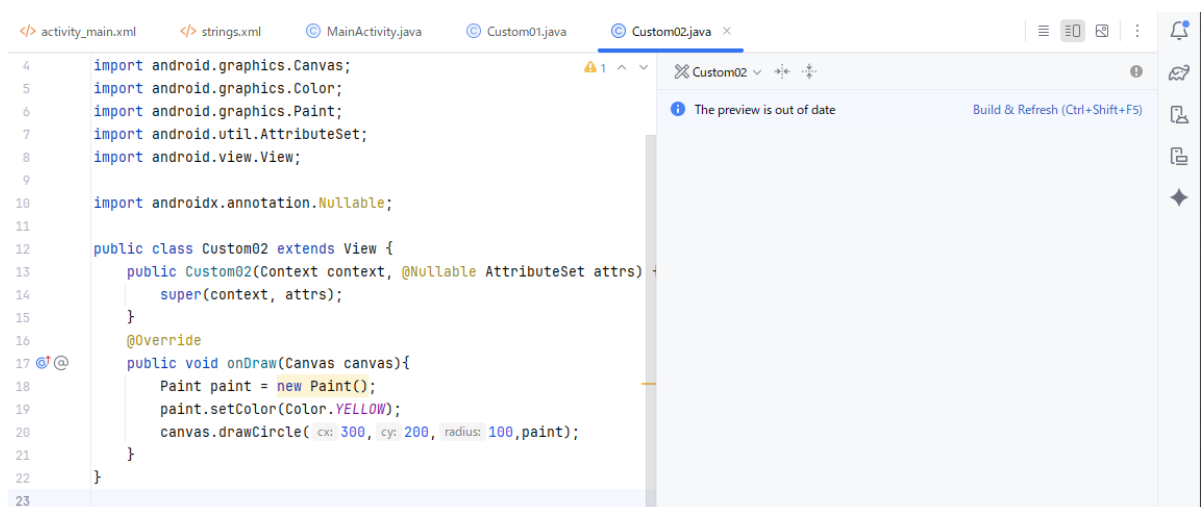
```
import androidx.annotation.Nullable;
```

```
public class Custom02 extends View {  
    public Custom02(Context context, @Nullable AttributeSet attrs) {  
        super(context, attrs);  
    }  
}
```

描画処理追加後↓

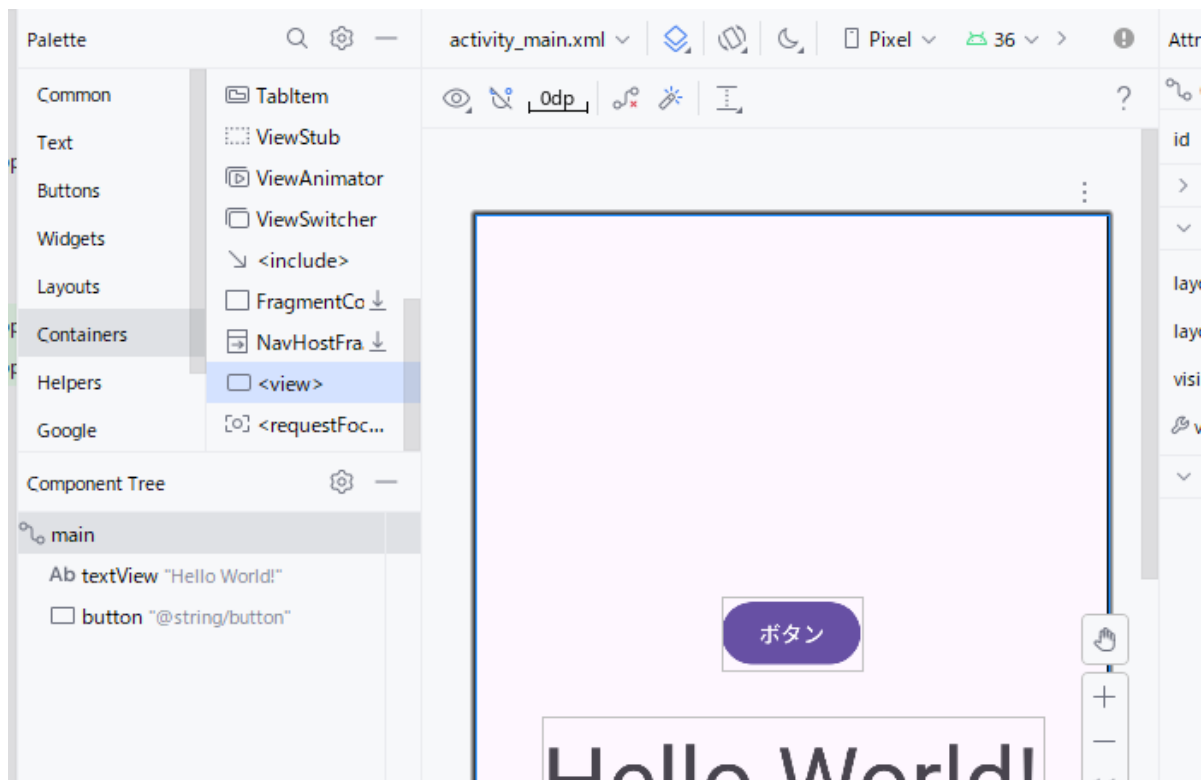
```
public class Custom02 extends View {  
    public Custom02(Context context, @Nullable AttributeSet attrs) {  
        super(context, attrs);  
    }  
  
    @Override //この行を追加  
    public void onDraw(Canvas canvas) { //この行を追加  
        Paint paint = new Paint(); //この行を追加  
        paint.setColor(Color.YELLOW); //この行を追加  
        canvas.drawCircle(300,200,100,paint); //この行を追加  
    } //この行を追加  
}
```

エディターの右上の 3 つのアイコンのうち一番左が選ばれていますが、真ん中に変えると以下の
のような画面になります。

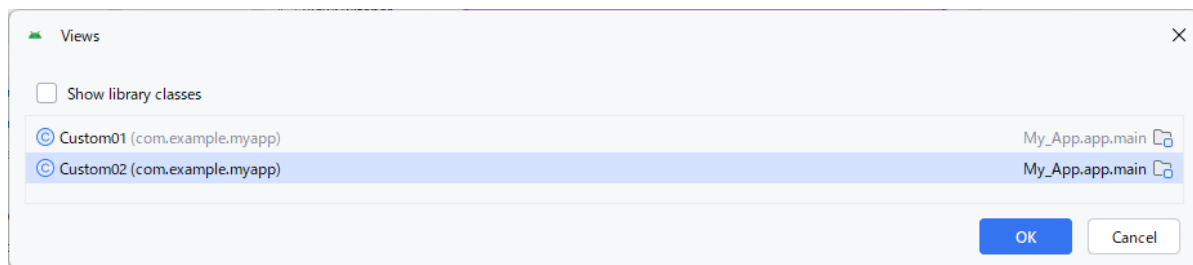


ここで右上の「Build & Refresh(Ctrl+Shift+F5)」を押しておきます。

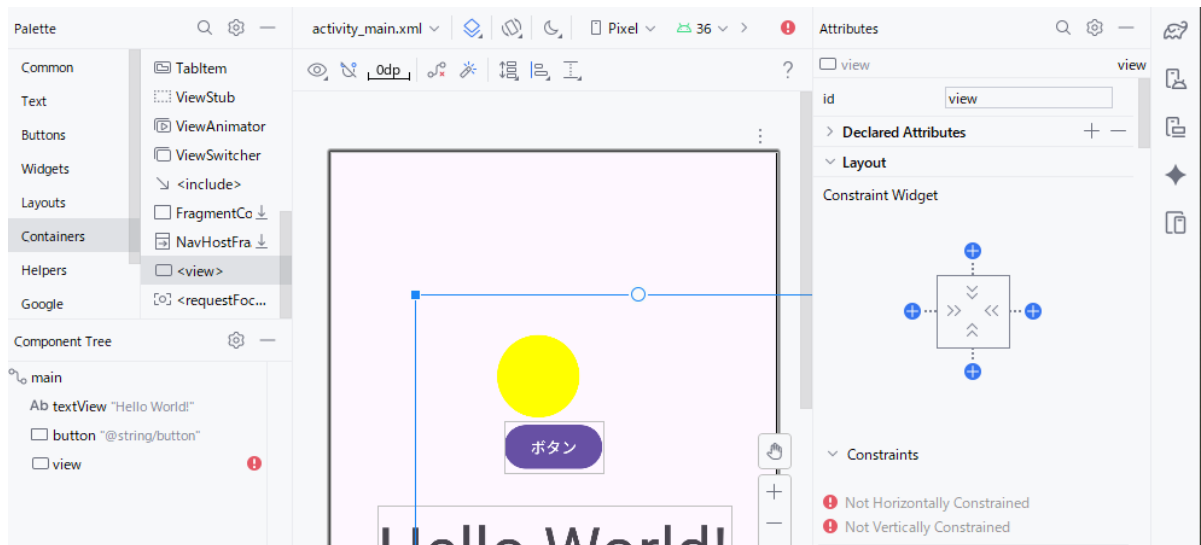
次に activity_main.xml のデザイン画面に映ります。まず Palette の左側の「Containers」をクリックして右側の部品をスクロールすると「<view>」というのがあります。これをドラッグ&ドロップでプレビュー画面に持っていきます。このとき位置は後で調整しますので気にしなくても大丈夫です。なるべく左上あたりが良いと思います。なお別のパレットの位置に似たものがありますので注意してください。



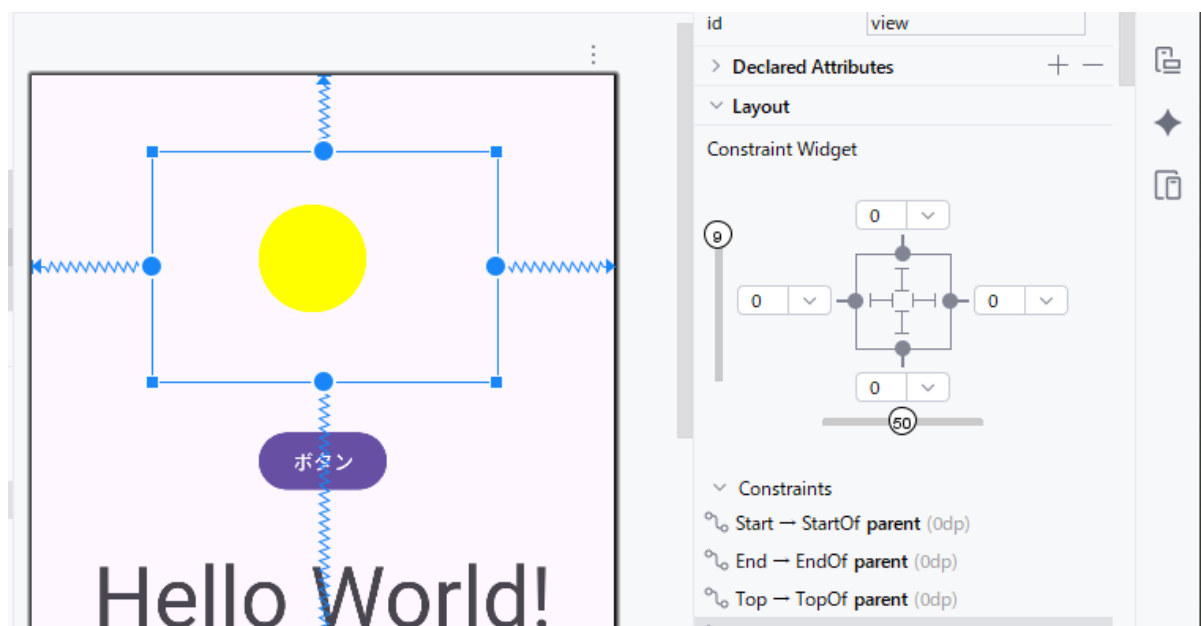
以下のようなダイアログが現れますのでここでは「Custom02」を選んで「OK」を押します。



これでカスタムビューと XML との連携がとれました。



赤いエラー部分は前述の通りですので Constraint の追加などを適切に施してください。右上の青い■で大きさを変えられます。全体を見るとこの■は右下や右上、左下にもあります。以下の図のようにすれば OK です。



なおカスタムビューのサイズは Custom02 クラスでは決めていませんので XML 側で決められた領域が描画領域になります。■をマウスで引っ張って好きな大きさにします。

次に画面に見えるボタンとカスタムビューとを機能的にむずびつけてみます。今 XML に追加された Custom02 は id 名が「view」になっています（図の右上）。これを使ってコード側でインスタンスを取得します。MainActivity クラスの onCreate メソッドの書き換えです。

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    //setContentView(new Custom01(this));           //この行をコメント
    Custom02 custom02 = findViewById(R.id.view);   //この行を追加
    Button button = findViewById(R.id.button);     //この行を追加
}
```

Custom02 に色を変えるメソッドを追加してみます。以下。

```
public class Custom02 extends View {
    int color;                                     //この行を追加
    public Custom02(Context context, @Nullable AttributeSet attrs) {
        super(context, attrs);
        color = Color.YELLOW;                     //この行を追加
    }
    public void changeColor() {                   //この行を追加
        color = Color.CYAN;                       //この行を追加
        invalidate();                             //この行を追加
    }                                              //この行を追加
    @Override
    public void onDraw(Canvas canvas) {
        Paint paint = new Paint();
        paint.setColor(color);                     //この行を変更（引数部分）
        canvas.drawCircle(300,200,100,paint);
    }
}
```

これで changeColor メソッドを呼び出せば黄色い●はシアン色の●に代わることになります。メソッド内の「invalidate()」は onDraw を暗示的に呼び出して描画を更新する役割を担っています。では MainActivity クラスにさらにコードを追加します。

```

protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    //setContentView(new Custom01(this));
    Custom02 custom02 = findViewById(R.id.view);
    Button button = findViewById(R.id.button);
    button.setOnClickListener(new View.OnClickListener() { //この行を追加
        @Override //この行を追加
        public void onClick(View view) { //この行を追加
            custom02.changeColor(); //この行を追加
        } //この行を追加
    }); //この行を追加
}

```

クリック時の処理部分は前述の入力補助機能をつかってください。なお追加部分は匿名クラスによるイベントハンドラーの追加ですが、XML によるイベントハンドラーを使う方法もあります。補足 B をご参照ください。

問題

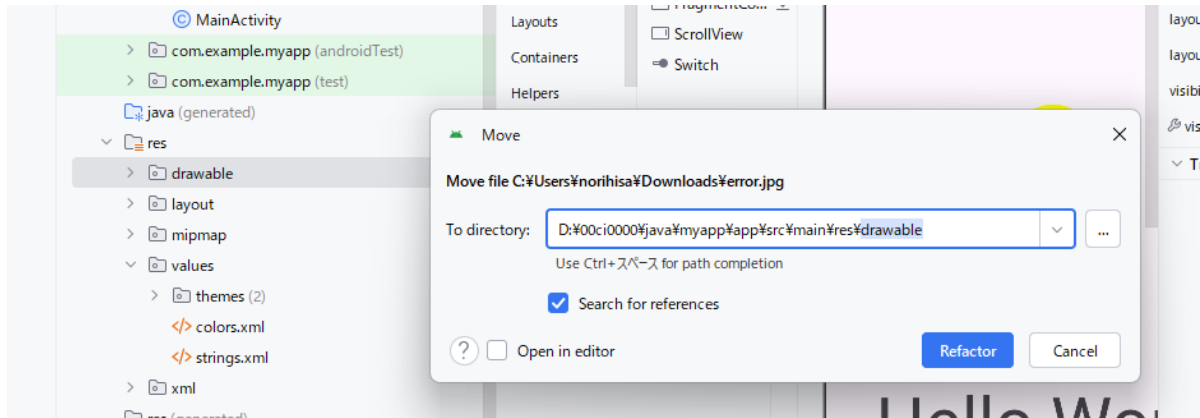
- ① クリックで●が大きくなるようにしてください。
- ② クリックで●が徐々に大きくなるようにしてください。
- ③ ボタンを 2 つ用意して一方をクリックすると●が徐々に大きく、もう一方をクリックすると●が徐々に小さくなるようにしてください。
- ④ 配列で色を 3 色格納しておき、ボタンを押すたびに順に色が変わるようにしてください。最後の色の次は最初の色になるようにしてください。
- ⑤ 色の名前を表示する Custom03 クラスを用意して、XML に貼り付け、上の問題の色の変化に合わせて色の名前も変わるようにしてみよ。この処理は本来 Custom02 に書いた方が効率いいのですが、ここでは練習のため、あえて指示通りにしてください。
- ⑥

10. カスタムビューの詳細 — 画像の読み込みと表示

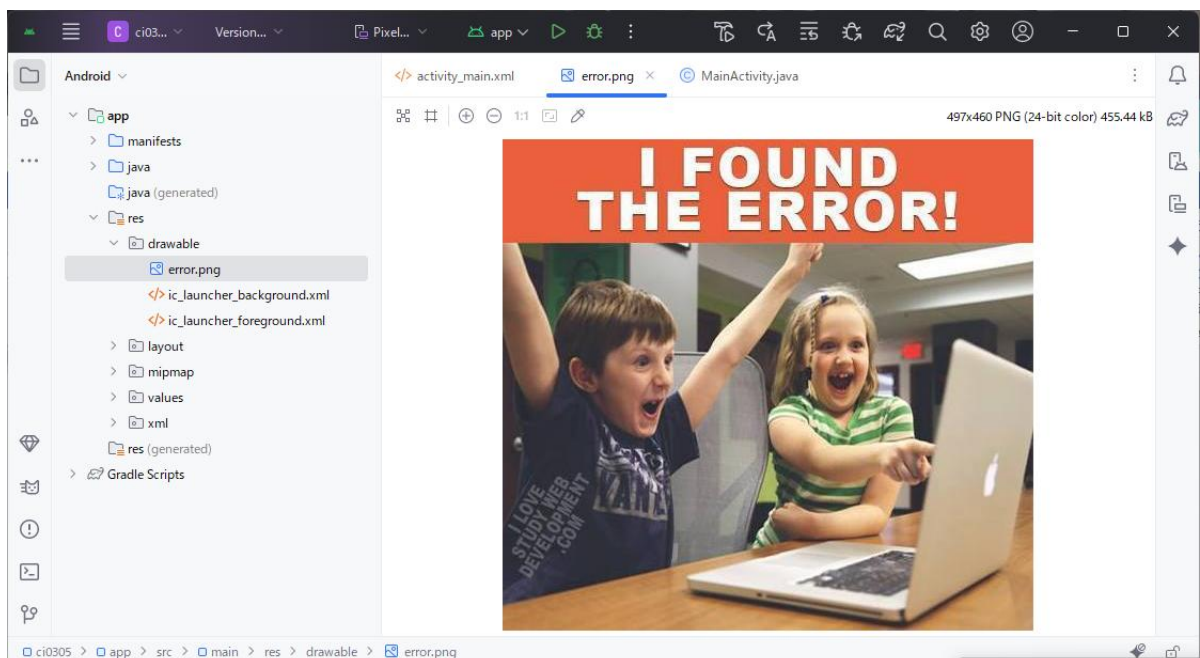
画像などの外部リソースは、プログラムコードの外にあり、読み込むためにその位置を特定したり、ない場合の処理をコードで記述する必要があります。ここでは内部リソースとして扱えるよう画像をプロジェクトに取り込んでみます。画像は jpg や png フォーマットが使えますが、ファイル名はアルファベット部分を小文字として先頭には数字が使えないといった制約がありま

す。使える記号は「_」のみです。ファイル名はそのままリソース名として自動的に割り振られて使われます。

画像をリソースとして登録するには画像ファイルをプロジェクトツリーの「res」→「drawable」へドラッグ&ドロップします。「Refactor」ボタンを押します。



こんな感じになっていれば OK です。図では「error.png」という画像を登録しましたのでリソース名は「error」です。画像を用意するときはファイル名が小文字であるかどうか、数字を先頭に使っていないかどうかをあらかじめ確認してからこの操作を行ってください。



もう一つの方法は直接プロジェクトフォルダへ画像をコピーするやり方になります。「プロジェクトフォルダ」→「app」→「src」→「main」→「res」→「drawable」へ画像をコピーすれば上

記の方法と同じ結果になります。

次に Custom01.java に画像リソースの読み込みと表示のコードを追加します。Bitmap のインポートが追加されることに注意してください。

```
public class Custom01 extends View {  
    Bitmap bitmap; //この行を追加  
    public Custom01(Context context) {  
        super(context);  
        bitmap = BitmapFactory.decodeResource(  
            getResources(), R.drawable.error); //この行を追加  
    }  
    @Override  
    public void onDraw(Canvas canvas) {  
        Paint paint = new Paint();  
        canvas.drawBitmap(bitmap, 100, 100, paint); //この行を追加  
    }  
}
```

実行結果：



XML 連携を行う場合は Custom02 に同じ操作を行ってください。Custom01 との違いはコンストラクタのみです。

問題

- ① 好きな画像を2つ用意して、2つの画像を重ならないように表示してみよ。

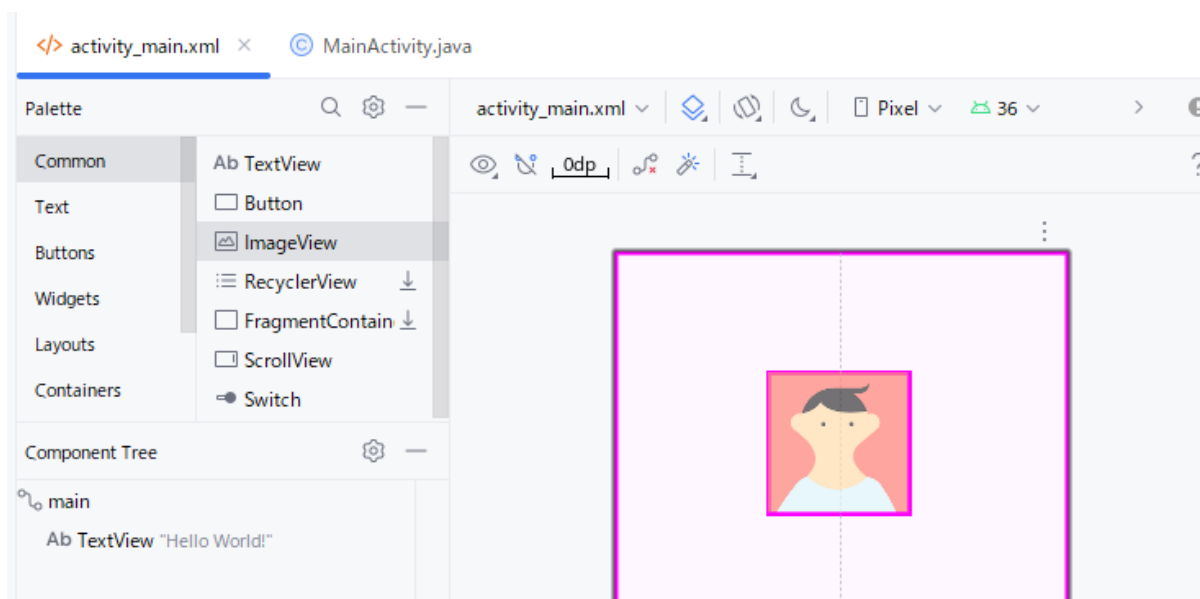
- ② 以下のコードを追加して Canvas が回転することを確認せよ。

```
Matrix matrix= new Matrix();  
matrix.setRotate(45);  
canvas.setMatrix(matrix);
```

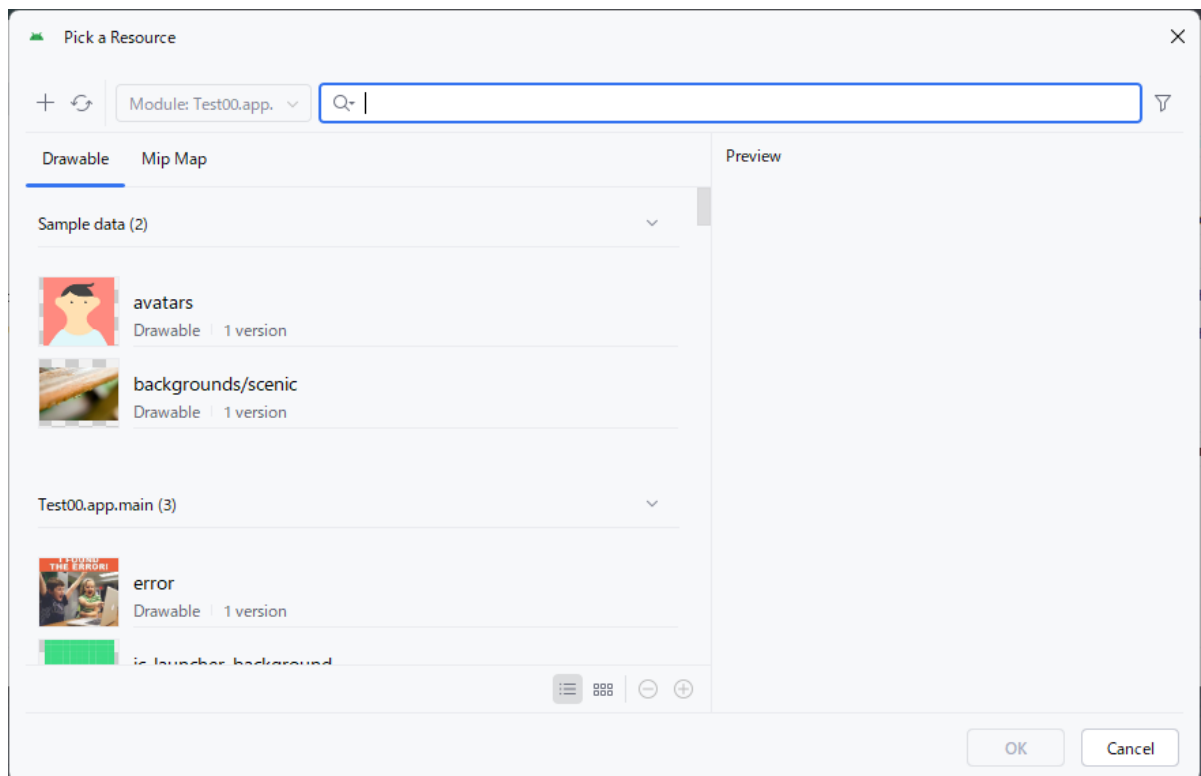
- ③ 2つの画像を異なるカスタムビューで表示させるようにして、ボタンを押すとそれぞれが逆方向に回るようにしてみよ。回転角は任意でよい。

1 1. ImageView を用いた画像の扱い

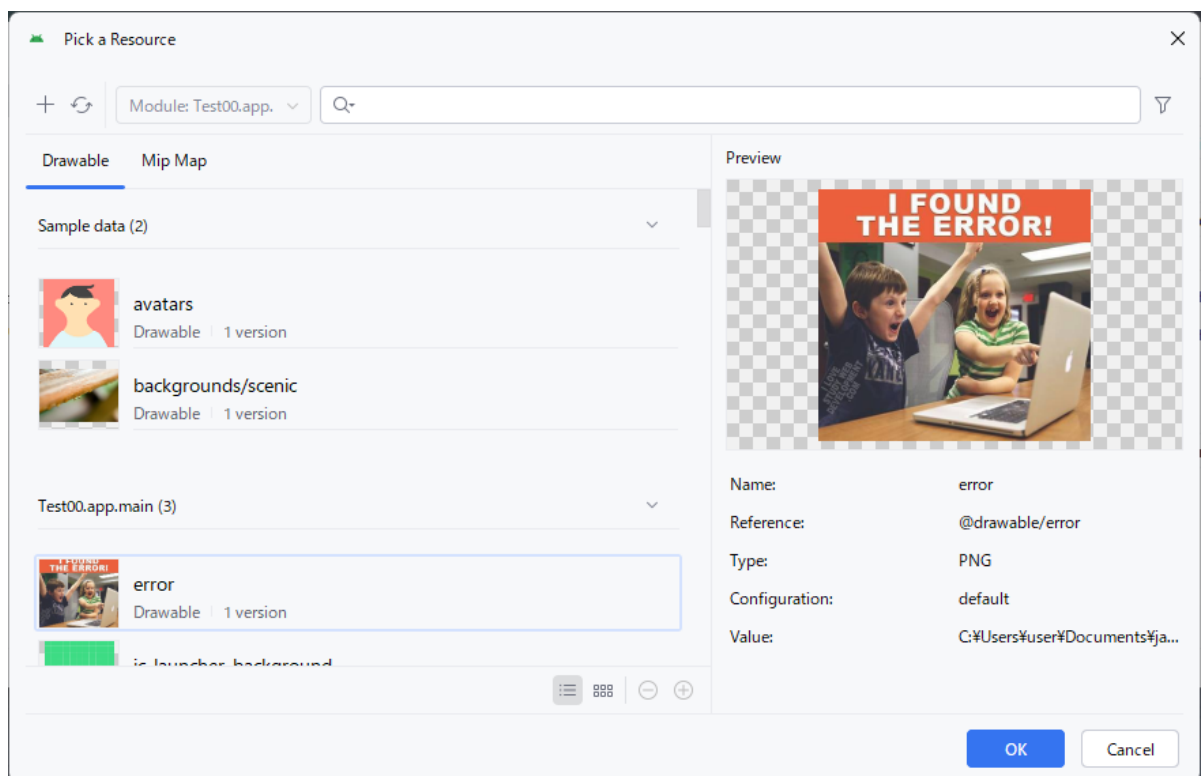
リソースに登録された画像は ImageView クラスで扱うことができます。XML で Palette から ImageView を見つけ、アイコンをプレビュー画面へドラッグ&ドロップします。



すると上記のようなアイコン表示が現れ、マウスのボタンを離すと以下のようなダイアログが現れます。その中にはリソースとして登録した画像が含まれているので、その画像をクリックします。

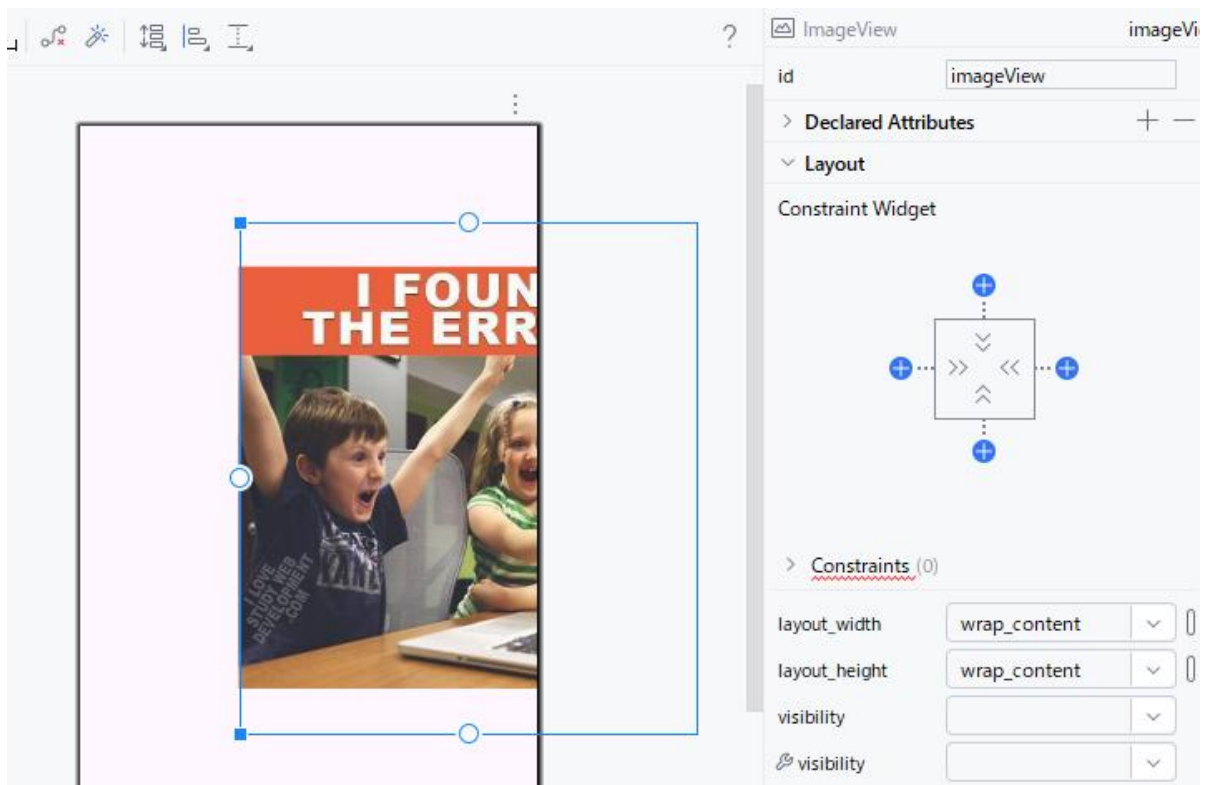


以下のように右側に画像のプレビューと情報が表示されるので「OK」を押します。

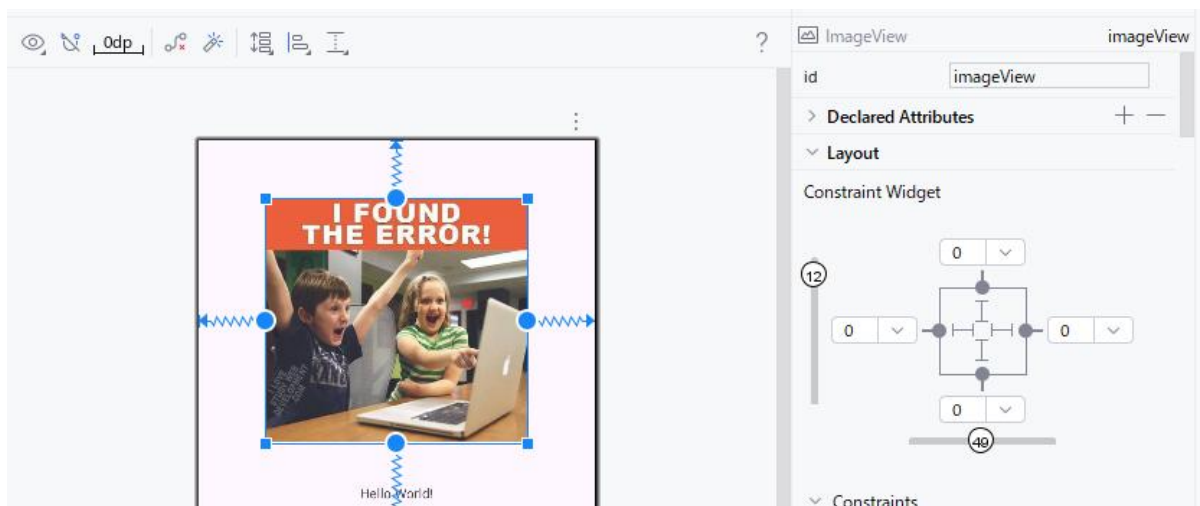


するとプレビューの方に画像が出ますので、あとは ConstraintLayout の紐づけを行います。サ

サイズ変更も併せて行います。



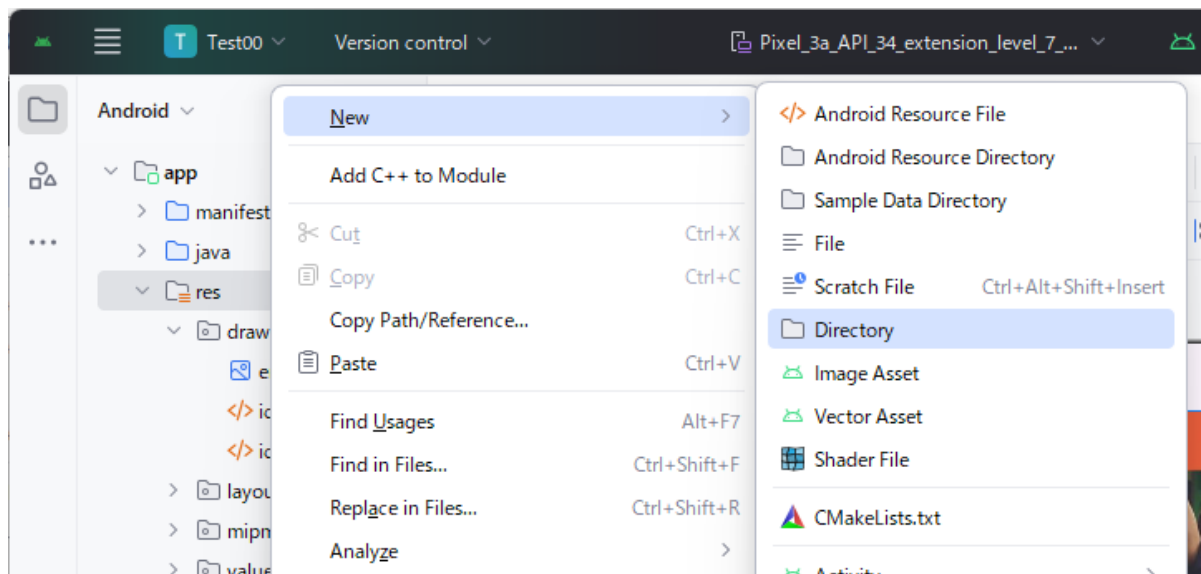
このようになれば OK です。



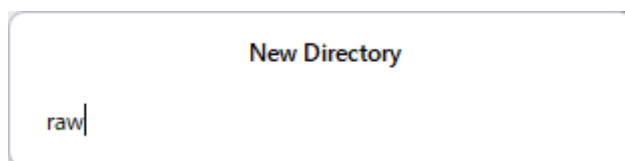
1 2. サウンドリソースの扱い

サウンドの再生はパレットに対応する View がないため自身で View を作るしかない。またリソースを格納するディレクトリがないので新規で res フォルダの直下に「raw」フォルダを作成してファイルをドラッグ&ドロップする。まず以下のように「res」フォルダを右クリックして「」

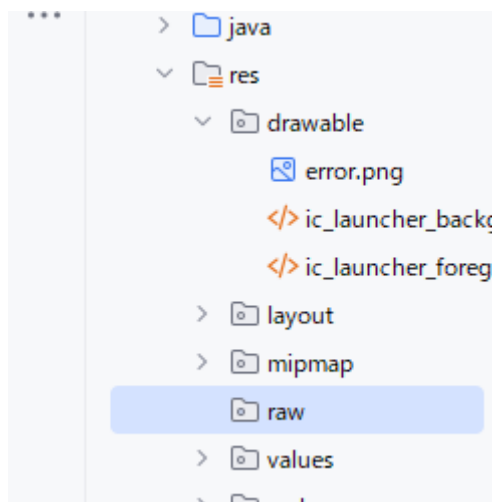
「New」 → 「Directory」 を選択します。



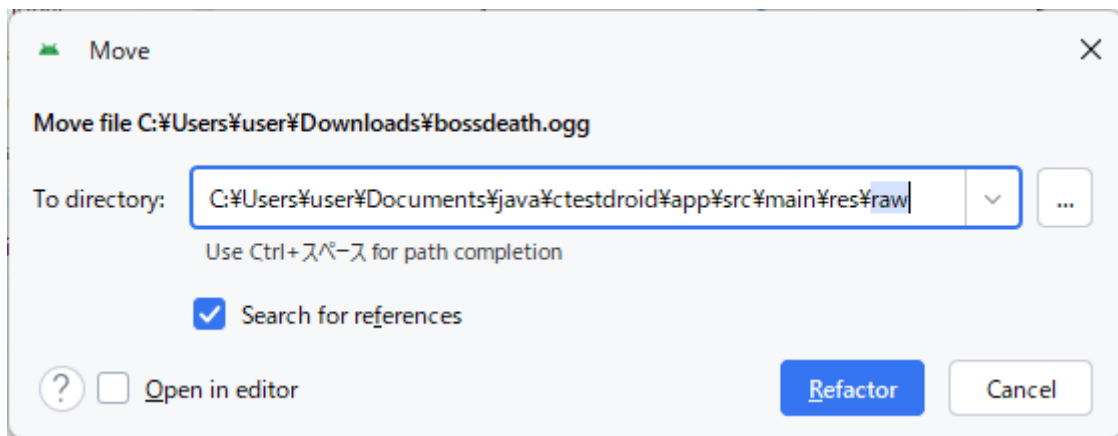
出てきたダイアログに「raw」と入力し、エンターを押します。



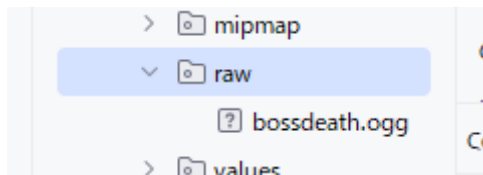
これで「res」の下に「raw」ができました。ここに音声データをドラッグ&ドロップします。



そうすると以下のダイアログが出ますのでそのまま「Refactor」ボタンを押します。



これでファイルがリソースに追加されました。ここでは「bossdeath.ogg」というファイルを使っています。



次に SoundView クラスを作成します。

```
import android.content.Context;
import android.media.AudioAttributes;
import android.media.SoundPool;
import android.view.View;

public class SoundView extends View {
    SoundPool pool;
    int poolMax;
    int id, id2;
    public SoundView(Context context) {
        super(context);
        poolMax = 2;
        AudioAttributes attr = new AudioAttributes.Builder()
            .setUsage(AudioAttributes.USAGE_MEDIA)
            .setContentType(AudioAttributes.CONTENT_TYPE_MUSIC)
            .build();
        pool = new SoundPool.Builder()
```

```

        .setAudioAttributes(attr).setMaxStreams(poolMax).build();
        id = pool.load(context,R.raw.bossdeath,1);
    }
    void play(){
        pool.play(id,1.0f,1.0f, 0, 0, 1);
    }
}

```

次に MainActivity クラスで SoundView クラスのインスタンスを生成して play メソッドを呼び出します。ここではボタンを用意してボタンを押したときに音が出るようにしています。

```

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Button b = findViewById(R.id.button);
        SoundView sv = new SoundView(this); //インスタンス生成
        b.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                sv.play(); //メソッドの呼び出し
            }
        });
    }
}

```

ボタンの追加の方法は 14～20 ページの記述を参照してください。

13. カスタムビューの詳細 — タッチイベント処理

onTouchEvent メソッドのオーバーライドをカスタムビューに記述するとカスタムビューがタッチイベントに反応ようになります。

```

public class Custom02 extends View {

```

```

int color;
float x, y;
public Custom02(Context context, @Nullable AttributeSet attrs) {
    super(context, attrs);
    color = Color.YELLOW;
}
public void changeColor() {
    color = Color.CYAN;
    invalidate();
}

@Override //この行を追加
public boolean onTouchEvent(MotionEvent event) { //この行を追加
    x = event.getX(); //この行を追加
    y = event.getY(); //この行を追加
    invalidate(); //この行を追加
    return super.onTouchEvent(event); //この行を追加
} //この行を追加

@Override
public void onDraw(Canvas canvas) {
    Paint paint = new Paint();
    paint.setColor(color);
    canvas.drawCircle(x, y, 100, paint);
}
}

```

これで黄色い●がタッチした位置に移動するようになります。

問題

- ① タッチするたびに色が変わるようにしてみよ。●は移動しなくてもよい。
- ② 画面の上半分をタッチすると●が赤、下半分をタッチすると青になるようにしてみよ。
なお画面の幅と高さはカスタムビューのコンストラクタ内で以下のようにすれば取れます。

```

Display display =
    ((Activity) context).getWindowManager().getDefaultDisplay();
Point point = new Point();

```

```
display.getSize(point);
w = point.x;           //幅
h = point.y;           //高さ
```

- ③ 画面の上半分をタッチすると画面全体が赤、下半分をタッチすると画面全体が青になるようにせよ。なお画面全体の塗りつぶしは drawColor メソッドを呼び出せばよい。

```
canvas.drawColor(Color.RED);
```

14. 描画ループ

カスタムビューの onDraw メソッド内で invalidate メソッドを呼び出すと、再帰的に onDraw メソッドが呼ばれるため描画処理がループするようになります。ただし onDraw メソッドを直接呼び出すわけではないのでいわゆる再帰処理にはならず、ディスプレイのリフレッシュレートに合わせて一定の時間間隔を置いて再帰的呼び出しが行われます。もしリフレッシュレートが 60Hz なら onDraw は 1 秒間に 60 回呼び出されます。

```
public class Custom03 extends View {
    float x, y;
    public Custom03(Context context) {
        super(context);
    }

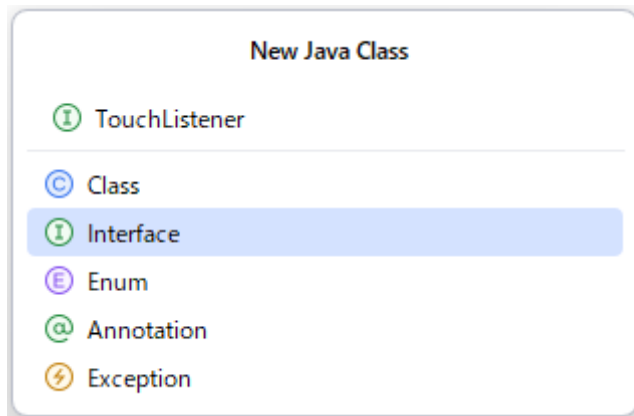
    @Override
    protected void onDraw(Canvas canvas) {
        super.onDraw(canvas);
        Paint paint = new Paint();
        canvas.drawCircle(x++, y++, 50, paint);
        invalidate();           //←これで再帰的呼び出しを行い繰り返される
    }
}
```

このカスタムビューの例では半径 50 の黒丸が右下に 1 秒間に 60 ピクセル移動します。

15. カスタムイベント

Button クラスには setOnClickListener メソッドがあり、引数に OnClickListener インターフェース

の匿名クラスを渡してボタンを押したときの処理が行えましたが、カスタムビューには `setOnXXXListener` などはないので、自作する必要があります。典型的な Listener パターンの用法になりますが、以下のようにして実現します。カスタムビューのタッチイベント用の Listener として、まず `TouchListener` インターフェースを次のようにしてみます。Interface はパッケージの右クリック→新規→Java クラスから以下のダイアログで下の 2 番目の Interface を選んでインターフェース名を書きます。



```
public interface TouchListener {  
    void isTouched();  
}
```

次に `Custom04.java` で以下のように Listener を実装します。

```
public class Custom04 extends View {  
    TouchListener touchListener;          //インターフェースのメンバ  
    public Custom04(Context context, @Nullable AttributeSet attrs) {  
        super(context, attrs);  
    }  
  
    void setOnTouchListener(TouchListener touchListener){  
        this.touchListener = touchListener;    //①  
    }  
  
    @Override  
    public boolean onTouchEvent(MotionEvent event) {  
        touchListener.isTouched();            //②  
        return super.onTouchEvent(event);  
    }  
}
```

```

    }

    @Override
    protected void onDraw(Canvas canvas) {
        super.onDraw(canvas);
        canvas.drawColor(Color.LTGRAY);
    }
}

```

コード内の②でインターフェースのメソッドを読んでいます。メソッドのオーバーライドは①に渡される引数の `TouchListener` のサブクラスに定義されるもので、呼び出し側の `MainActivity` クラスで匿名クラスで定義します。以下のように匿名クラスを `MainActivity` に書きます。

```

protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    //setContentView(new Custom01(this));
    Custom02 custom02 = findViewById(R.id.view);
    Custom04 custom04 = findViewById(R.id.view2);
    custom04.setOnTouchListener(new TouchListener() {
        @Override
        public void isTouched() {
            custom02.changeColor();
        }
    });
}

```

この例では上で紹介している `Custom02` クラスの `changeColor` メソッドを、`Custom04` の範囲をタッチしたときに呼び出すようにしています。以下の実行結果のように下部に `Custom04` を配置し（灰色部分）、タッチすると上部の `Custom02` の●の色が変わるようにしています。これでボタンを使った時の `setOnTouchListener` メソッドに書いた処理とほぼ同じになりました。



16. アクティビティの切り替え

アクティビティはマニフェストで起動時のアクティビティを決めるので、AndroidManifest.xmlの中を書き換えることで異なるアクティビティを使うことができます。以下の「ここ」と書かれた部分です。

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapplication">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MyApplication">
        <activity
            android:name=".MainActivity"           ←ここ
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>

```

起動後はアクティビティを切り替えることはできませんが、前述の通り、MainActivity クラスで setContentView メソッドを呼び出しているので、これを使って MainActivity で使用する View を切り替えることができます。以下は activity_main.xml と activity_sub を切り替える例です。main メソッドでは setContentView メソッドで activity_main.xml を読み込んでその XML 内のオブジェクトの記述をし、sub メソッドでは逆に activity_sub.xml を読み込んでその XML 内のオブジェクトを記述します。Create メソッドでは最初に表示する main メソッドを呼び出しておき、activity_main.xml 内のボタンを押すと sub メソッドが呼び出されて activity_sub.xml に入れ替わるという仕組みです。

```

public class MainActivity extends Activity {
    void main() {
        setContentView(R.layout.activity_main);
    }
}

```

```

        Button button = findViewById(R.id.button);
        button.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                sub();
            }
        });
    }

    void sub() {
        setContentView(R.layout.activity_sub);
        Button button = findViewById(R.id.button2);
        button.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                main();
            }
        });
    }

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        main();
    }
}

```

他に Intent クラスを使う方法や、非推奨となった Fragment を使う方法などいくつかの選択肢はありますが、ここでは割愛します。

17. その他の View クラスの詳細

- ① CalenderView の用例
- ② WebView の用法
- ③ CheckBox の用法
- ④ RadioButton, RadioGroup の用法
- ⑤ Spinner の用法

1 8.

1 9. 補足 A

```
package com.example.myapplication;

import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;
import androidx.constraintlayout.widget.ConstraintLayout;
import androidx.constraintlayout.widget.ConstraintSet;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        //setContentView(R.layout.activity_main);

        Button button = new Button(this);
        button.setText("Hello");
        button.setId(View.generateViewId());
        TextView view = new TextView(this);
        view.setText("World");
        ConstraintLayout layout = new ConstraintLayout(this);
        layout.addView(button);
        ConstraintLayout.LayoutParams layoutParams
            = new ConstraintLayout.LayoutParams(
                ConstraintLayout.LayoutParams.WRAP_CONTENT,
                ConstraintLayout.LayoutParams.WRAP_CONTENT
            );
        button.setLayoutParams(layoutParams);
        ConstraintSet constraintSet = new ConstraintSet();
        constraintSet.clone(layout);
        constraintSet.connect(button.getId(), ConstraintSet.START,
            ConstraintSet.PARENT_ID, ConstraintSet.START);
        constraintSet.connect(button.getId(), ConstraintSet.END,
```

```

        ConstraintSet.PARENT_ID, ConstraintSet.END);
constraintSet.connect(button.getId(), ConstraintSet.TOP,
        ConstraintSet.PARENT_ID, ConstraintSet.TOP);
constraintSet.connect(button.getId(), ConstraintSet.BOTTOM,
        ConstraintSet.PARENT_ID, ConstraintSet.BOTTOM);
constraintSet.setHorizontalBias(button.getId(), 0.0f);
constraintSet.setVerticalBias(button.getId(), 0.05f);
constraintSet.applyTo(layout);
layout.addView(view);
setContentView(layout);

    }
}

```

20. 補足 B

XML 側のイベントハンドラを使う方法

MainActivity.java に以下のようなメソッドを追加

```

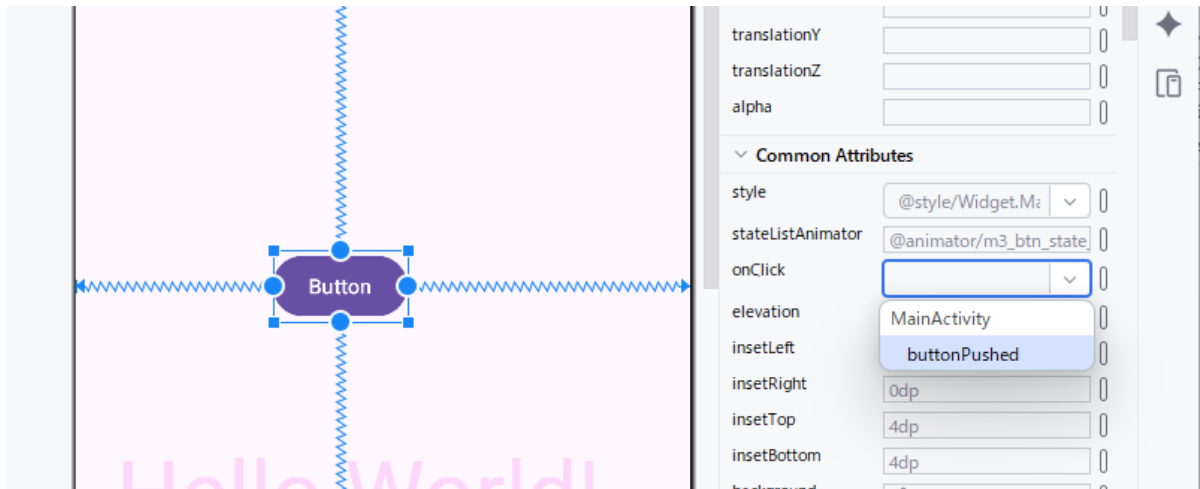
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    public void buttonPushed() { //追加部分
                                //追加部分
    }
                                //追加部分
}

```

activity_main.xml の方で以下のように onClick に buttonPushed があれば、選択することでボタンを押したときに MainActivity.java に記述した buttonPushed メソッドが呼ばれます。



うまくこの選択項目が出てこない場合は直接 XML を編集します。矢印部分を追加して下さい。



例えば以下のように MainActivity.java を書き足すと、ボタンを押したときにボタンのラベルが「こんにちは」に変わります。

```
import android.widget.Button; //追加部分

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    Button b; //追加部分
    @Override
    protected void onCreate(Bundle savedInstanceState) {
```



```
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
        b=findViewById(R.id.button);           //追加部分  
    }  
    public void buttonPushed(View view){  
        b.setText("こんにちは");           //追加部分  
    }  
}
```

2 1.